

DOCUMENT DE CONCEPTION ET MAINTENANCE

Application de Gestion du Matériel
IUT GEII - Département Génie Electrique

Cadre : SAE Bases de Données - BUT3 GEII

Etablissement : IUT Lyon 1 - Département GEII

Date : Janvier 2026

Auteur : Ben Abdeslam Sofia, Chaalane Badice, Samadi Chahd, Hemici Sefana

Encadrant : P. Bouron & G. Robin

Lien du site publié et opérationnel : <https://sisterlike-interbranchial-chace.ngrok-free.dev>

Table des matières

Table des matières.....	2
PARTIE 1 : CAHIER DES CHARGES	5
1.1 Contexte du Projet	5
1.1.1 Présentation du Département GEII	5
1.1.2 Le Parc Matériel à Gérer	5
1.1.3 Les Utilisateurs du Système	6
1.2 Situation Actuelle et Problématiques.....	7
1.2.1 Gestion Manuelle Actuelle	7
1.2.2 Problèmes Identifiés	7
1.2.3 Impact sur le Département	8
1.3 Objectifs du Projet.....	9
1.3.1 Objectifs Fonctionnels	9
1.3.2 Objectifs Techniques	10
1.3.3 Bénéfices Attendus - Synthèse Chiffree	10
1.4 Fonctionnalités - État de Réalisation	11
1.4.1 Tableau Exhaustif des 42 Fonctionnalités	11
1.4.2 Récapitulatif par Module	18
1.5 Fonctionnalités Non Implémentées (Evolutions Futures).....	19
1.5.1 Evolutions Futures Planifiées.....	19
1.5.2 Priorisation et Roadmap	20
PARTIE 2 : ARCHITECTURE DE L'APPLICATION	22
2.1 Présentation des Différentes Pages (Formulaires)	22
2.1.1 Page d'Authentification (index.html).....	22
2.1.2 Dashboard Administrateur (dashboard_admin.html).....	26
2.1.3 Dashboard Utilisateur (dashboard_user.html).....	30
2.1.4 Dashboard Technicien (dashboard_technicien.html)	31
2.2 Documentation de l'API REST	32
2.2.1 Vue d'Ensemble de l'API	32
2.2.2 Endpoints Authentification (/api/auth/*)	32
2.2.3 Endpoints Matériel (/api/matériels/*)	34
2.2.4 Endpoints Utilisateurs (/api/users/*)	35
2.2.5 Endpoints Emprunts (/api/mes-emprunts/*)	35

2.2.6 Endpoints Maintenances (/api/maintenances/*)	35
2.2.7 Endpoints Statistiques et Exports	35
2.3 Description du Découpage en Dossiers/Fichiers	36
2.3.1 Structure Globale du Projet	36
2.3.2 Description Détaillée des Fichiers Principaux	37
2.4 Modélisation de la Base de Données	39
2.4.1 Modèle Entité-Association (E/A)	39
2.4.2 Schema Relationnel	40
2.4.3 Description Détaillée des Tables SQL	41
2.4.4 Vues SQL	44
2.4.5 Triggers SQL	45
PARTIE 3 : MAINTENANCE ET EVOLUTIVITE	47
3.1 Procédures de Sauvegarde	47
3.1.1 Importance des Sauvegardes	47
3.1.2 Stratégie de Sauvegarde (3-2-1)	47
3.1.3 Commandes de Sauvegarde MySQL	48
3.1.4 Procédures de Restauration	50
3.2 Maintenance Applicative	51
3.2.1 Gestion des Logs.....	51
3.2.2 Mises à Jour de l'Application	52
3.2.3 Monitoring et Surveillance	53
3.3 Evolutions Futures et Roadmap	55
3.3.1 Evolutions Court Terme (v1.1 - T1 2026).....	55
3.3.2 Evolutions Moyen Terme (v2.0 - T2-T3 2026).....	55
3.3.3 Evolutions Long Terme (v3.0 - 2027).....	55
3.3.4 Roadmap du Projet.....	56
Autoévaluation	57
Note proposée : 17.5/ 20	57
Répartition des contributions	57
ANNEXES	58
Annexe A : Glossaire Technique.....	58
A.1 Termes Généraux	58
A.2 Termes Techniques	58
A.3 Termes Metier (Application)	59

Annexe B : Comptes de Test	60
B.1 Liste des Comptes de Test.....	60
B.2 Permissions par Rôle	60
B.3 Scenarios de Test Recommandes	60
Annexe C : Contacts et Support.....	62
C.1 Equipe Projet	62
C.2 Support Technique.....	62
C.3 Ressources Utiles	63
C.4 Demandes d'Evolution	63

PARTIE 1 : CAHIER DES CHARGES

1.1 Contexte du Projet

1.1.1 Présentation du Département GEII

Le Département GEII (Génie Electrique et Informatique Industrielle) de l'IUT Lyon 1 forme chaque année environ 150 étudiants aux métiers de l'électricité, de l'électronique et de l'automatisme industriel. Cette formation technique nécessite l'utilisation intensive de matériel spécialisé pour les travaux pratiques, les projets tutorés et les stages.

Le département dispose de plusieurs salles de travaux pratiques équipées de matériel électronique de haute précision : oscilloscopes numériques, générateurs de signaux, alimentations stabilisées, multimètres, analyseurs de spectre, automates programmables industriels (API), et de nombreux composants électroniques.

C'est dans ce contexte que la présence d'une plateforme d'emprunt peut s'avérer extrêmement utile.

1.1.2 Le Parc Matériel à Gérer

Le parc matériel du département GEII représente un investissement conséquent qui doit être géré avec rigueur. Ce matériel est reparti dans plusieurs localisations et utilise par différentes catégories d'utilisateurs.

Inventaire détaillé par catégorie :

Type matériel	Qte	Catégorie	Marques
Oscilloscopes numériques	25	Mesure	Tektronix, Rigol, Keysight
Générateurs de signaux	20	Mesure	Agilent, Rigol
Alimentations stabilisées	40	Mesure	TTi, Tenma, GW Instek
Multimètres numériques	60	Mesure	Fluke, Agilent, Keysight
Analyseurs de spectre	5	Mesure	Rigol, Siglent
Automates API	15	Automatisme	Siemens, Schneider, Allen-Bradley
Variateurs de vitesse	10	Automatisme	ABB, Schneider
Cartes Arduino/ESP32	100	Embarque	Arduino, Espressif
Raspberry Pi	30	Embarque	Raspberry Pi Foundation
PC portables TP	20	Informatique	Dell, HP, Lenovo
Composants électroniques	500+	Composants	Divers fournisseurs

Ce matériel est utilisé quotidiennement pour les séances de travaux pratiques (environ 30 heures par semaine) et est également emprunté par les étudiants pour leurs projets personnels, les projets tutorés de BUT2/BUT3, et les stages.

1.1.3 Les Utilisateurs du Système

L'application de gestion du matériel est destinée à plusieurs types d'utilisateurs avec des besoins et des droits d'accès différents. Cette catégorisation permet de sécuriser l'application et d'adapter l'interface à chaque profil.

Les 4 profils utilisateurs :

1. Administrateur (Admin)

Rôle : Gestionnaire principal du système avec tous les droits.

Utilisateurs concernés : Responsable du département, secrétariat, responsable technique.

Nombre estime : 2-3 personnes.

Droits : Gestion complète des utilisateurs, matériel, emprunts, maintenances, statistiques, exports.

2. Technicien

Rôle : Responsable de la maintenance et des réparations du matériel.

Utilisateurs concernés : Techniciens de laboratoire.

Nombre estime : 3 personnes.

Droits : Gestion des maintenances, consultation matériel, création emprunts pour tests.

3. Enseignant

Rôle : Utilisateur principal pour les emprunts de matériel pédagogique.

Utilisateurs concernés : Enseignants, enseignants-chercheurs, vacataires.

Nombre estime : 20-25 personnes.

Droits : Consultation matériel, création/gestion de ses propres emprunts, historique personnel.

4. Élève

Rôle : Emprunteur de matériel pour projets étudiants.

Utilisateurs concernés : Etudiants BUT1, BUT2, BUT3.

Nombre estime : 150+ personnes.

Droits : Consultation matériel disponible, demande d'emprunt (validation requise), historique personnel.

Tableau récapitulatif des droits par profil :

Fonctionnalite	Admin	Tech	Enseignant	Élève
Gestion utilisateurs	Oui	Non	Non	Non
Ajout/Modification matériel	Oui	Oui	Non	Non
Suppression matériel	Oui	Non	Non	Non
Création emprunt (tous)	Oui	Oui	Oui	Non
Création emprunt (soi-même)	Oui	Oui	Oui	Oui*
Validation emprunt	Oui	Oui	Non	Non
Gestion maintenances	Oui	Oui	Non	Non
Accès statistiques	Oui	Partiel	Non	Non
Export Excel/PDF	Oui	Oui	Non	Non

1.2 Situation Actuelle et Problématiques

1.2.1 Gestion Manuelle Actuelle

Avant le développement de cette application, la gestion du matériel du département GEII était effectuée de manière entièrement manuelle, avec plusieurs méthodes coexistant sans coordination :

Méthode 1 : Fichiers Excel disperses

Chaque technicien ou enseignant responsable d'une salle maintenait son propre fichier Excel avec l'inventaire du matériel. Ces fichiers n'étaient pas partagés et chacun utilisait un format différent (colonnes, nommage, catégories). Résultat : impossibilité d'avoir une vue globale du parc matériel.

Méthode 2 : Cahier d'emprunts papier

Un cahier papier était placé dans chaque salle de TP pour noter les emprunts. L'emprunteur devait inscrire son nom, la date, le matériel emprunté et la date de retour prévue. Ce système présentait de nombreuses failles : écriture illisible, oubli de noter, pages arrachées, cahier perdu ou emporté.

Méthode 3 : Communication informelle

Les emprunts de courte durée se faisaient souvent par simple accord oral entre l'emprunteur et le technicien. Aucune trace écrite n'était conservée, rendant impossible tout suivi ou rappel en cas de non-retour.

1.2.2 Problèmes Identifiés

L'analyse de la situation existante a permis d'identifier 5 problématiques majeures qui impactent directement le fonctionnement du département :

Problème 1 : Absence de centralisation des données

Les informations sont dispersées dans des fichiers Excel multiples, des cahiers papier et la mémoire des personnes. Chaque enseignant/technicien a sa propre méthode de suivi sans standardisation. Il n'existe pas de vue d'ensemble du parc matériel permettant de répondre à des questions simples comme 'Combien d'oscilloscopes sont disponibles aujourd'hui ?'

Conséquence : Perte de temps considérable (2-3h/semaine) pour chercher l'information, risque d'erreurs et de doublons.

Problème 2 : Traçabilité des emprunts insuffisante

Les emprunts notés sur papier ou dans des tableurs Excel non-partagés ne permettent pas de connaître l'historique fiable des emprunts passés. Il est difficile de savoir qui détient quel matériel à un instant T, et depuis combien de temps. Les dates de retour prévues ne sont pas respectées car aucun système de rappel n'existe.

Conséquence : Matériel égaré ou 'oublié', conflits d'emprunts quand plusieurs personnes ont besoin du même équipement, absence de responsabilisation des emprunteurs.

Problème 3 : Suivi des maintenances inexistant

Aucun système ne permet de planifier les maintenances préventives (calibration annuelle des oscilloscopes, vérification des alimentations, etc.). Les réparations sont notées manuellement sans

suivi centralise. Les couts des maintenances ne sont pas traces, rendant impossible l'analyse du cout total de possession d'un matériel.

Conséquence : Matériel qui tombe en panne de manière imprévue pendant les TP, budget maintenance mal géré, impossibilité de planifier les remplacements.

Problème 4 : Gestion des utilisateurs rudimentaire

Il n'existe pas de système de droits d'accès différenciés. N'importe qui peut théoriquement emprunter n'importe quel matériel sans validation. Impossible de limiter l'accès à certains matériels sensibles ou coûteux (ex: analyseur de spectre à 8000 EUR). Pas de validation des emprunts par un responsable.

Conséquence : Risque de vols ou de dégradations, utilisation abusive de matériel coûteux, absence de responsabilité claire.

Problème 5 : Absence totale de statistiques

Impossible de répondre à des questions stratégiques : Quel matériel est le plus utilisé ? Quels sont les pics d'emprunts dans l'année ? Quel est le taux de disponibilité moyen ? Ces données sont pourtant essentielles pour justifier les demandes de budget auprès de l'IUT et planifier les achats futurs.

Conséquence : Décisions d'investissement prises à l'aveugle, impossibilité de négocier des achats groupés, pas de visibilité sur les tendances d'utilisation.

1.2.3 Impact sur le Département

Quantification des impacts :

Type d'impact	Fréquence	Volume annuel	Nature
Temps perdu recherche info	2-3 heures/semaine	~100h/an	Cout indirect
Matériel non localisable	5-10% du parc	~20 items	Risque financier
Conflits d'emprunts	2-3 par mois	~30/an	Friction organisationnelle
Pannes imprévues en TP	1-2 par mois	~15/an	Impact pédagogique
Retards emprunts non détectés	~20% des emprunts	-	Désorganisation

1.3 Objectifs du Projet

Face aux problématiques identifiées, le projet vise à développer une application web complète permettant de centraliser et automatiser la gestion du matériel. Les objectifs sont classés en deux catégories : fonctionnels et techniques.

1.3.1 Objectifs Fonctionnels

Objectif 1 : Centraliser toutes les données matérielles

Créer une base de données unique et partagée accessible en temps réel par tous les utilisateurs autorisés. Toutes les informations sur le matériel seront standardisées : nom, description, numéro de série unique, catégorie, état (neuf/bon/usage/défectueux/hs), localisation, date d'acquisition, photos.

Bénéfice attendu : Gain de temps de 80% dans la recherche d'informations. Réponse instantanée à 'Où est tel matériel ?'

Objectif 2 : Tracer 100% des emprunts

Mettre en place un système d'emprunts complet avec : historique de qui a emprunté quoi et quand, dates de retour prévues et effectives, statuts automatiques (en_cours, en_retard, termine), alertes email automatiques en cas de retard, blocage des emprunts si retards existants.

Bénéfice attendu : 100% de traçabilité, responsabilisation des emprunteurs, réduction de 90% des matériels 'perdus'.

Objectif 3 : Gérer les maintenances

Permettre la planification des maintenances préventives (calibrations annuelles, vérifications périodiques) et le suivi des réparations (date début, date fin, technicien assigné, coût, description). Conserver l'historique complet des interventions par matériel.

Bénéfice attendu : Réduction de 30% des pannes imprévues, meilleure gestion du budget maintenance, prolongation de la durée de vie du matériel.

Objectif 4 : Gérer les utilisateurs avec droits différenciés

Implémenter 4 profils utilisateurs (Admin, Technicien, Enseignant, Élève) avec des permissions adaptées à chaque rôle. Mettre en place une authentification sécurisée avec hachage bcrypt des mots de passe et vérification email obligatoire.

Bénéfice attendu : Sécurité renforcée, limitation des abus, responsabilité clairement établie pour chaque action.

Objectif 5 : Fournir des statistiques et indicateurs

Créer un Dashboard avec KPI en temps réel : nombre de matériels disponibles, emprunts en cours, maintenances actives, retards. Générer des graphiques d'évolution (emprunts par mois, répartition par catégorie, états du parc).

Bénéfice attendu : Aide à la décision basée sur données réelles, justification factuelle des demandes de budget.

Objectif 6 : Permettre des exports multiformats

Proposer l'export des données en format Excel (.xlsx) pour analyses externes et en PDF pour rapports imprimables. Permettre le filtrage des données avant export (par catégorie, état, période).

Bénéfice attendu : Compatibilité avec outils existants (Excel, PowerPoint), facilite de reportant pour la direction.

1.3.2 Objectifs Techniques

Objectif technique	Critère de succès
Performance	Temps de réponse < 1 seconde pour toutes les requêtes API
Sécurité	Hashage bcrypt (cout 12), protection injection SQL, sessions sécurisées
Disponibilité	Application accessible 24/7 via navigateur web
Compatibilité	Fonctionne sur PC, tablette, smartphone (design responsive)
Maintenabilité	Code commenté, architecture MVC, séparation frontend/backend
Evolutivité	API REST permettant futures intégrations (mobile, IoT)

1.3.3 Bénéfices Attendus - Synthèse Chiffree

Indicateur	Avant	Après (objectif)	Gain
Temps recherche information	2-3h/semaine	15-30 min/semaine	-80%
Matériel non localisable	5-10%	<1%	-90%
Pannes imprévues	1-2/mois	0-1/mois	-50%
Retards non détectés	~20%	<5%	-75%
Temps création rapport	2-3h	5 min (export auto)	-95%

1.4 Fonctionnalités - État de Réalisation

Cette section présente le tableau exhaustif des 42 fonctionnalités de l'application, organisées par module. Pour chaque fonctionnalité, on indique : l'identifiant unique, la description complète, la priorité, le statut d'implémentation et des commentaires techniques détaillés.

Légende des statuts :

FAIT = Fonctionnalité 100% implémentée et testée

PARTIEL = Fonctionnalité partiellement implémentée (backend OK, frontend à compléter)

NON FAIT = Fonctionnalité non implémentée (évolution future)

1.4.1 Tableau Exhaustif des 42 Fonctionnalités

MODULE : AUTHENTIFICATION (F01 - F05)

[F01] Inscription utilisateur

Formulaire avec nom, prénom, email, password. Validation stricte (format email, complexité password 8+ caractères avec majuscule/minuscule/chiffre/spécial). Envoi email avec code 6 chiffres. Hash bcrypt du mot de passe (cout 12). Insertion en base avec email_verifie=0.

Priorité : Critique | Statut : **FAIT**

Implémentation : POST /api/register. Code app.py lignes 301-380. Email via Flask-Mail SMTP.

[F02] Vérification email

Saisie code 6 chiffres reçu par email. Vérification code + expiration (10 min). Si valide : email_verifie=1, activation compte. Si invalide/expire : message erreur avec possibilité renvoi.

Priorité : Critique | Statut : **FAIT**

Implémentation : POST /api/verify-email. Sécurité : code expire 10 min, évite faux comptes.

[F03] Connexion

Formulaire email + password. Vérification : email existe + actif=1 + email_verifie=1. Comparaison password bcrypt.checkpw(). Si OK : création session Flask + redirection selon rôle.

Priorité : Critique | Statut : **FAIT**

Implémentation : POST /api/login. Redirection dynamique selon rôle utilisateur.

[F04] Déconnexion

Destruction session Flask. Suppression cookie session. Redirection vers page login. Nettoyage localStorage cote client.

Priorité : Critique | Statut : **FAIT**

Implémentation : POST /api/logout. Simple mais essentiel pour sécurité.

[F05] Mot de passe oublié

Formulaire email. Génération token unique + envoi email reset. Lien valide 30 min. Page saisie nouveau password avec validation complexité.

Priorité : Moyenne | Statut : **PARTIEL**

Implémentation : Backend implémenté (lignes 420-480) mais frontend reset_password.html manquant.

MODULE : GESTION UTILISATEURS - ADMIN (F06 - F10)

[F06] Liste utilisateurs

Affichage tableau tous utilisateurs : nom, prénom, email, rôle, statut actif, date création. Tri par colonne cliquable. Pagination (10 par page). Recherche temps réel par nom/email.

Priorité : Critique | Statut : **FAIT**

Implémentation : GET /api/users. Frontend dashboard_admin.html section #users.

[F07] Ajouter utilisateur

Modal formulaire : nom, prénom, email, password, rôle (select). Validation client (regex email, password). POST au backend. Vérification unicité email. Hash password + insertion.

Priorité : Critique | Statut : **FAIT**

Implémentation : POST /api/users. Admin peut créer comptes sans vérification email.

[F08] Modifier utilisateur

Modal prérempli avec données existantes. Modification nom, prénom, rôle. Password optionnel (si vide = inchange). PUT au backend avec update en base.

Priorité : Critique | Statut : **FAIT**

Implémentation : PUT /api/users/:id. Si password fourni : re-hash bcrypt.

[F09] Supprimer utilisateur

Confirmation modale. Vérification cascade : si user à emprunts en cours -> erreur bloquante. Sinon suppression physique de l'enregistrement.

Priorité : Critique | Statut : **FAIT**

Implémentation : DELETE /api/users/:id. Protection intégrité référentielle.

[F10] Activer/Désactiver utilisateur

Toggle switch dans tableau. PUT au backend. Update actif=1 ou 0. Si désactivé : user ne peut plus se connecter mais données conservées (soft delete).

Priorité : Importante | Statut : **FAIT**

Implémentation : PUT /api/users/:id/toggle-status. Preserve historique emprunts.

MODULE : GESTION MATÉRIEL (F11 - F19)

[F11] Liste matériels

Affichage grille cartes avec : photo principale, nom, numéro série, catégorie, état (badge colore), localisation, disponibilité. Tri (nom, catégorie, état). Design responsive : 4 colonnes desktop, 2 tablettes, 1 mobile.

Priorité : Critique | Statut : **FAIT**

Implémentation : GET /api/matériel. Photos depuis /static/uploads/.

[F12] Détails matériel

Modal avec toutes infos : nom, description, numéro série, catégorie, état, localisation, date acquisition. Galerie photos (carousel si plusieurs). Historique 5 derniers emprunts. Historique 5 dernières maintenances.

Priorité : Importante | Statut : **FAIT**

Implémentation : GET /api/matériel/:id. Vue SQL vue_matériel_complet.

[F13] Ajouter matériel

Modal formulaire : nom (text), description (textarea), numéro série (unique), catégorie (select), état (select 5 valeurs), localisation, date acquisition (datepicker). Upload jusqu'à 5 photos (max 5Mo chacune).

Priorité : Critique | Statut : **FAIT**

Implémentation : POST /api/matériel. Validation unicité numéro série.

[F14] Modifier matériel

Modal prérempli. Modification tous champs sauf numéro série (non modifiable). Upload photos supplémentaires. Suppression photos existantes (icone croix).

Priorité : Critique | Statut : **FAIT**

Implémentation : PUT /api/matériel/:id. Numéro série non modifiable (historique).

[F15] Supprimer matériel

Confirmation modale. Vérification : si emprunts en cours -> erreur. Si maintenances en cours -> erreur. Sinon suppression cascade (photos, emprunts passés, maintenances).

Priorité : Critique | Statut : **FAIT**

Implémentation : DELETE /api/matériel/:id. CASCADE SQL + os.remove() photos.

[F16] Upload photos

Input file accept=image/* multiple. Validation client : max 5 fichiers, max 5Mo chacun. Preview avant upload. Nom sécurisé : {matériel_id}_{timestamp}_{nom}.jpg. Stockage /static/uploads/.

Priorité : Importante | Statut : **FAIT**

Implémentation : POST /api/matériel/:id/photos. secure_filename() sécurité.

[F17] Supprimer photo

Icone croix sur chaque photo. Confirmation. DELETE backend. Suppression enregistrement photos_matériel + fichier disque physique.

Priorité : Moyenne | Statut : **FAIT**

Implémentation : DELETE /api/photos/:id. Vérification appartenance matériel.

[F18] Recherche temps réel

Input search dans barre. Evènement keyup JavaScript avec debounce 300ms. Filtrage cote client. Recherche dans : nom, description, numéro série, catégorie, localisation.

Priorité : Importante | Statut : **FAIT**

Implémentation : Fonction filterMaterials() dans admin.js. toLowerCase().

[F19] Filtres avancés

Selects : catégorie (liste dynamique), état (5 valeurs ENUM), disponibilité (checkbox). Combinaison filtres (ET logique). Bouton reset filtres.

Priorité : Importante | Statut : **FAIT**

Implémentation : Filtrage client après chargement. GET /api/catégories.

MODULE : GESTION EMPRUNTS (F20 - F26)

[F20] Liste emprunts

Tableau avec : matériel (nom + numéro série), emprunteur (nom + prénom), date emprunt, date retour prévue, date retour effective, statut (badge colore : vert=termine, orange=en_cours, rouge=en_retard). Tri par colonne. Filtres statut/emprunteur.

Priorité : Critique | Statut : **FAIT**

Implémentation : GET /api/emprunts. Statut calcule automatiquement.

[F21] Créer emprunt

Modal formulaire : matériel (select uniquement disponibles), emprunteur (select users actifs), date retour prévue (datepicker min=aujourd'hui), commentaire optionnel. Vérification disponibilité matériel. Update disponible=0.

Priorité : Critique | Statut : **FAIT**

Implémentation : POST /api/emprunts. date_emprunt = NOW() auto.

[F22] Valider retour

Bouton Retour sur ligne emprunt. Confirmation modale. Update : date_retour_effective=NOW(), statut='termine'. Update matériel.disponible=1. Toast notification.

Priorité : Critique | Statut : **FAIT**

Implémentation : PUT /api/emprunts/:id/return. Calcul durée réelle.

[F23] Modifier emprunt

Modal prérempli. Modification : date retour prévue (prolongation), commentaire. Emprunteur/matériel non modifiables (cohérence historique).

Priorité : Moyenne | Statut : **FAIT**

Implémentation : PUT /api/emprunts/:id. Date modifiable si statut != termine.

[F24] Supprimer emprunt

Confirmation modale. DELETE backend. Si statut='en_cours' : update matériel.disponible=1 (libération). Suppression enregistrement.

Priorité : Moyenne | Statut : **FAIT**

Implémentation : *DELETE /api/emprunts/:id. Permet annuler erreur.*

[F25] Alertes retards automatiques

Script scheduler_emails.py en continu. Chaque jour 8h : SELECT emprunts en retard. Pour chaque : UPDATE statut='en_retard' + envoi email emprunteur + email admin.

Priorité : Importante | Statut : **FAIT**

Implémentation : *Library schedule. Flask-Mail envoi. Template HTML.*

[F26] Mes emprunts (vue user)

Page spécifique utilisateurs non-admin. Affichage uniquement LEURS emprunts. Filtres : en cours, terminés, en retard. Sécurité : pas d'accès emprunts autres users.

Priorité : Importante | Statut : **FAIT**

Implémentation : *GET /api/emprunts/user/:id. Vérification session.*

MODULE : GESTION MAINTENANCES (F27 - F32)

[F27] Liste maintenances

Tableau : matériel (nom + numéro série), type (préventive/corrective/améliorative), technicien assigné, description, date début, date fin, statut (badge : bleu=planifiée, orange=en_cours, vert=terminée, gris=annulée), coût.

Priorité : Importante | Statut : **FAIT**

Implémentation : *GET /api/maintenances. Filtres statut/technicien/type.*

[F28] Créer maintenance

Modal : matériel (select), type (select 3 valeurs), technicien (select rôle='technicien'), description, date début, coût estimé. Statut initial='planifiée'.

Priorité : Importante | Statut : **FAIT**

Implémentation : *POST /api/maintenances. Validation date future.*

[F29] Démarrer maintenance

Bouton Démarrer si statut='planifiée'. Confirmation. Update : statut='en_cours', date_début=NOW(). TRIGGER : update matériel.disponible=0.

Priorité : Importante | Statut : **FAIT**

Implémentation : *PUT /api/maintenances/:id action='start'. Trigger SQL.*

[F30] Terminer maintenance

Bouton Terminer si statut='en_cours'. Modal saisie coût réel. Update : statut='terminée', date_fin=NOW(), coût=X. TRIGGER : update matériel.disponible=1.

Priorité : Importante | Statut : **FAIT**

Implémentation : *PUT /api/maintenances/:id action='finish'. Calcul durée.*

[F31] Annuler maintenance

Bouton Annuler si planifiée ou en_cours. Confirmation + raison. Update statut='annulée'. Si était en_cours : TRIGGER remet disponible=1.

Priorité : Moyenne | Statut : **FAIT**

Implémentation : PUT /api/maintenances/:id action='cancel'. Raison stockée.

[F32] Mes maintenances (technicien)

Vue filtrée pour techniciens : uniquement maintenances assignées. Tri par statut (en_cours prioritaire). Actions rapides. Badge notification si retards.

Priorité : Importante | Statut : **FAIT**

Implémentation : GET /api/maintenances/technicien/:id. Dashboard technicien.

MODULE : STATISTIQUES ET DASHBOARD (F33 - F36)

[F33] Dashboard KPI

4 cartes statistiques : Total matériels (COUNT), Disponibles (COUNT WHERE disponible=1), Emprunts en cours (COUNT statut='en_cours'), Maintenances actives. Refresh automatique 30 secondes. Animation compteur.

Priorité : Importante | Statut : **FAIT**

Implémentation : GET /api/stats/dashboard. Requête SQL optimisées.

[F34] Graphique emprunts/mois

Chart.js line chart. 12 derniers mois. Données : COUNT emprunts GROUP BY MONTH. Axe X=mois, Y=nombre. Hover détails. Couleur ligne bleu IUT.

Priorité : Importante | Statut : **FAIT**

Implémentation : GET /api/stats/monthly. Chart.js responsive + tooltips.

[F35] Graphique matériel/catégorie

Chart.js doughnut (camembert). Données : COUNT matériel GROUP BY catégorie. Légende avec pourcentages. Palette 8 couleurs. Click segment -> filtre liste.

Priorité : Importante | Statut : **FAIT**

Implémentation : GET /api/stats/matériel/catégories. Couleurs custom.

[F36] Graphique états matériel

Chart.js bar horizontal. 5 barres (neuf, bon, usage, défectueux, hs). Données : COUNT GROUP BY état. Code couleur vert->rouge selon gravité.

Priorité : Importante | Statut : **FAIT**

Implémentation : GET /api/stats/matériel/états. Couleurs sémantiques.

MODULE : EXPORTS (F37 - F38)

[F37] Export Excel

Bouton Export Excel. Génération fichier .xlsx avec openpyxl. Colonnes : toutes infos matériels. Styles : en-têtes bleu + blanc, filtres auto, colonnes ajustées. Téléchargement direct (Content-Disposition: attachment). Cleanup fichier après.

Priorité : Importante | Statut : **FAIT**

Implémentation : GET /api/export/excel. Fichier matériels_{date}.xlsx.

[F38] Export PDF

Bouton Export PDF. Génération PDF avec reportlab. Tableau formate avec bordures. En-tête logo + titre. Pied de page : date + numéro page. Format A4 paysage. Téléchargement direct.

Priorité : Importante | Statut : **FAIT**

Implémentation : GET /api/export/pdf. Font Arial. Stockage temporaire.

MODULE : INTERFACE UTILISATEUR (F39 - F42)

[F39] Thème clair/sombre

Toggle bouton dans header (icone soleil/lune). Click -> swap CSS variables (--bg-primary, --text-primary, etc.). Sauvegarde préférence localStorage. Chargement auto au retour. Transition smooth 0.3s.

Priorité : Basse | Statut : **FAIT**

Implémentation : *thème.js. document.documentElement.setAttribute('data-thème').*

[F40] Notifications toast

Système notifications visuelles. 4 types : success (vert), error (rouge), warning (orange), info (bleu). Apparition slide-in top-right. Durée configurable (défaut 3s). Icones Lucide. Stack si multiple.

Priorité : Moyenne | Statut : **FAIT**

Implémentation : *toast.js. Méthodes .success(), .error(), .warning(), .info().*

[F41] Design responsive

Breakpoints : mobile (<768px), tablette (768-1024px), desktop (>1024px). Mobile : sidebar collapse burger, grid 1 colonne, forms full-width. Tablette : grid 2 colonnes. Desktop : 3-4 colonnes.

Priorité : Importante | Statut : **FAIT**

Implémentation : *Media queries CSS. Sidebar hamburger. Touch events.*

[F42] Sidebar navigation

Menu latéral fixe 260px. Sections : Dashboard, Matériel, Emprunts, Maintenances (selon rôle), Utilisateurs (admin). Toggle thème en bas. Bouton déconnexion. Collapse sur mobile.

Priorité : Importante | Statut : **FAIT**

Implémentation : *Layout flexbox. Icons Lucide. Active state CSS.*

1.4.2 Récapitulatif par Module

Module	IDs	Total	Fait	Partiel	Non fait	Avancement
Authentification	F01-F05	5	4	1	0	80%
Gestion Utilisateurs	F06-F10	5	5	0	0	100%
Gestion Matériel	F11-F19	9	9	0	0	100%
Gestion Emprunts	F20-F26	7	7	0	0	100%
Gestion Maintenances	F27-F32	6	6	0	0	100%
Statistiques	F33-F36	4	4	0	0	100%
Exports	F37-F38	2	2	0	0	100%
Interface Utilisateur	F39-F42	4	4	0	0	100%
TOTAL	F01-F42	42	41	1	0	98%

1.5 Fonctionnalités Non Implémentées (Evolutions Futures)

Cette section liste les fonctionnalités qui n'ont pas été implémentées dans la version actuelle du projet, mais qui constituent des évolutions intéressantes pour les versions futures. Ces fonctionnalités ont été identifiées lors de l'analyse des besoins mais n'ont pas été retenues dans le périmètre initial.

1.5.1 Evolutions Futures Planifiées

[FN01] API REST avec JWT

Priorité : Moyenne | Complexité estimée : 2-3 semaines

Endpoints API accessibles depuis applications mobiles natives avec authentification JWT (JSON Web Token). Permettrait le développement d'une app mobile indépendante.

Raison non-implémentation : *Hors périmètre projet. Nécessite refonte complétée du système d'authentification actuel basé sur sessions Flask.*

[FN02] Notifications push temps réel

Priorité : Importante | Complexité estimée : 3 semaines

WebSockets pour notifications instantanées : nouvel emprunt, maintenance assignée, retard détecté. L'utilisateur recevrait une notification dans le navigateur sans recharger la page.

Raison non-implémentation : *Complexité technique WebSocket. Nécessite serveur Node.js ou Flask-SocketIO en parallèle.*

[FN03] QR codes matériel

Priorité : Moyenne | Complexité estimée : 1-2 semaines

Génération QR code unique par matériel. Scan QR avec smartphone -> affichage fiche matériel + possibilité emprunt rapide. Utile pour inventaire physique.

Raison non-implémentation : *Nécessite library qrcode (Génération) + intégration scan frontend HTML5 Camera API.*

[FN04] Import CSV/Excel en masse

Priorité : Basse | Complexité estimée : 1 semaine

Upload fichier .csv ou .xlsx pour créer matériels en masse (ex: 100 matériels d'un coup). Interface mapping colonnes. Validation ligne par ligne.

Raison non-implémentation : *Fonctionnalité 'nice to have' mais pas critique. Ajout manuel suffisant pour parc actuel (~200 items).*

[FN05] Système de réservations

Priorité : Importante | Complexité estimée : 2-3 semaines

Réserver matériel à l'avance (calendrier). Visualisation disponibilités. Validation réservation par admin. Gestion conflits de réservations.

Raison non-implémentation : *Complexité logique importante (conflits, notifications, annulations). Nécessite refonte modèle emprunts.*

[FN06] Historique audit complet

Priorité : Moyenne | Complexité estimée : 1 semaine

Logger toutes actions utilisateurs (qui a modifié quoi quand). Table audit_logs. Interface consultation logs pour admin. Export logs.

Raison non-implémentation : *Charge serveur + stockage significatifs. Pas exige pour projet BUT.*

[FN07] Dashboard BI avance

Priorité : Basse | Complexité estimée : 1-2 mois

Intégration Power BI ou Tableau pour analyses poussées. KPI complexes. Prévisions basées sur historique (machine learning).

Raison non-implémentation : *Hors compétences projet BUT. Nécessite serveur BI dédié + licences couteuses.*

[FN08] Application mobile native

Priorité : Basse | Complexité estimée : 2-3 mois

App iOS/Android native (React Native ou Flutter). Scan QR, notifications push, géolocalisation matériel emprunte.

Raison non-implémentation : *Hors périmètre. Progressive Web App (responsive actuel) suffit pour usage courant.*

[FN09] Signature électronique

Priorité : Basse | Complexité estimée : 1-2 semaines

Signature numérique emprunteur lors création emprunt (canvas HTML5 ou DocuSign). Preuve juridique d'engagement.

Raison non-implémentation : *Complexité juridique (valeur légale ?). Pas nécessaire dans contexte IUT (confiance établie).*

[FN10] Interface multi-langue

Priorité : Basse | Complexité estimée : 1 semaine

Interface en FR/EN. Détection automatique langue navigateur. Fichiers i18n JSON pour traductions.

Raison non-implémentation : *Public uniquement francophone (IUT Lyon). Aucun besoin identifié.*

1.5.2 Priorisation et Roadmap

Pourquoi ces fonctionnalités ne sont pas implémentées :

Contrainte temps : Le projet s'inscrit dans le cadre d'une SAE avec un volume horaire limite (3 séances encadrées de 3H30 + travail autonome). L'effort a été concentré sur les fonctionnalités critiques du cahier des charges.

Contrainte compétences : Certaines technologies (WebSocket, React Native, Machine Learning) sont hors du programme BUT3 GEII et nécessiteraient une formation supplémentaire.

Contrainte périmètre : Le cahier des charges initial était centré sur la gestion basique du matériel (inventaire, emprunts, maintenances). Les fonctionnalités avancées relèvent d'évolutions futures.

Principe MVP (Minimum Viable Product) : L'approche adoptée est de livrer d'abord une version fonctionnelle avec les essentiels, puis d'itérer avec des améliorations.

Roadmap proposée :

Horizon	Fonctionnalité	Justification
Court terme (v1.1)	FN03 - QR codes	Utilité immédiate pour inventaire physique
Court terme (v1.1)	FN05 - Mot de passe oublié (frontend)	Compléter F05 partiel
Moyen terme (v2.0)	FN05 - Réservations	Demande fréquente des enseignants
Moyen terme (v2.0)	FN02 - Notifications push	Amélioration UX significative
Long terme (v3.0)	FN01 - API JWT	Prérequis pour app mobile
Long terme (v3.0)	FN08 - App mobile	Complément à l'app web

PARTIE 2 : ARCHITECTURE DE L'APPLICATION

2.1 Présentation des Différentes Pages (Formulaires)

L'application web est composée de 4 pages HTML principales, chacune adaptée à un profil utilisateur spécifique. Cette section décrit en détail chaque page, ses composants, ses formulaires et son fonctionnement.

2.1.1 Page d'Authentification (index.html)

Rôle de la page : Porte d'entrée de l'application. Première page affichée lors de l'accès à l'URL. Gere l'authentification des utilisateurs : connexion, inscription et vérification email.

URL d'accès : <http://localhost:5000/> ou <http://localhost:5000/index.html>

Formulaire 1 : Connexion

Ce formulaire permet aux utilisateurs existants de se connecter à l'application. Il est affiché par défaut lors du chargement de la page.

Champ	Type HTML	Type	Requis	Placeholder	Validation
Email	input	email	Oui	votre.email@iut.fr	Regex validation email
Mot de passe	input	password	Oui	-	Min 6 caractères
Se connecter	button	submit	-	-	Appel POST /api/auth/login

Comportement du formulaire :

1. L'utilisateur saisit son email et mot de passe
2. Clic sur 'Se connecter' déclenche la validation frontend
3. Si validation OK : POST /api/auth/login avec JSON {email, password}
4. Si réponse 200 : stockage user dans localStorage + redirection selon rôle
5. Si erreur : affichage toast avec message d'erreur



Figure 1: Formulaire de connexion

Logique de redirection selon le rôle :

Rôle	Page de redirection	Description
admin	dashboard_admin.html	Accès complet a toutes les fonctionnalités
technicien	dashboard_technicien.html	Gestion des maintenances assignées
enseignant	dashboard_user.html	Emprunts et consultation matériel
élève	dashboard_user.html	Emprunts et consultation matériel

Formulaire 2 : Inscription

Ce formulaire permet la création de nouveaux comptes utilisateurs. Il est accessible via le lien 'S'inscrire' sous le formulaire de connexion.

Champ	Type HTML	Type	Requis	Placeholder	Validation
Nom	input	text	Oui	Dupont	Max 100 caractères
Prénom	input	text	Oui	Jean	Max 100 caractères
Email	input	email	Oui	jean.dupont@iut.fr	Regex + unicité BDD
Mot de passe	input	password	Oui	-	Min 6 car, complexité
Confirmer MDP	input	password	Oui	-	Doit être identique
Rôle	select	enum	Oui	-	élève, enseignant, technicien
S'inscrire	button	submit	-	-	POST /api/auth/register

Critères de validation du mot de passe :

- Minimum 6 caractères
(configurable dans config.py)
- Recommande : 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial
- Indicateur visuel de force (barre de progression)

IUT Lyon 1 - GEII
Gestion du Matériel

Inscription

Nom * Prénom *

Entrez votre nom Entrez votre prénom

Email *

Entrez votre email universitaire

Mot de passe * Confirmer *

6 caractères minimum Confirmez le mot de passe

Rôle *

– Sélectionner –

S'INSCRIRE

Déjà un compte ? [Se connecter](#)

Figure 2: Formulaire d'inscription

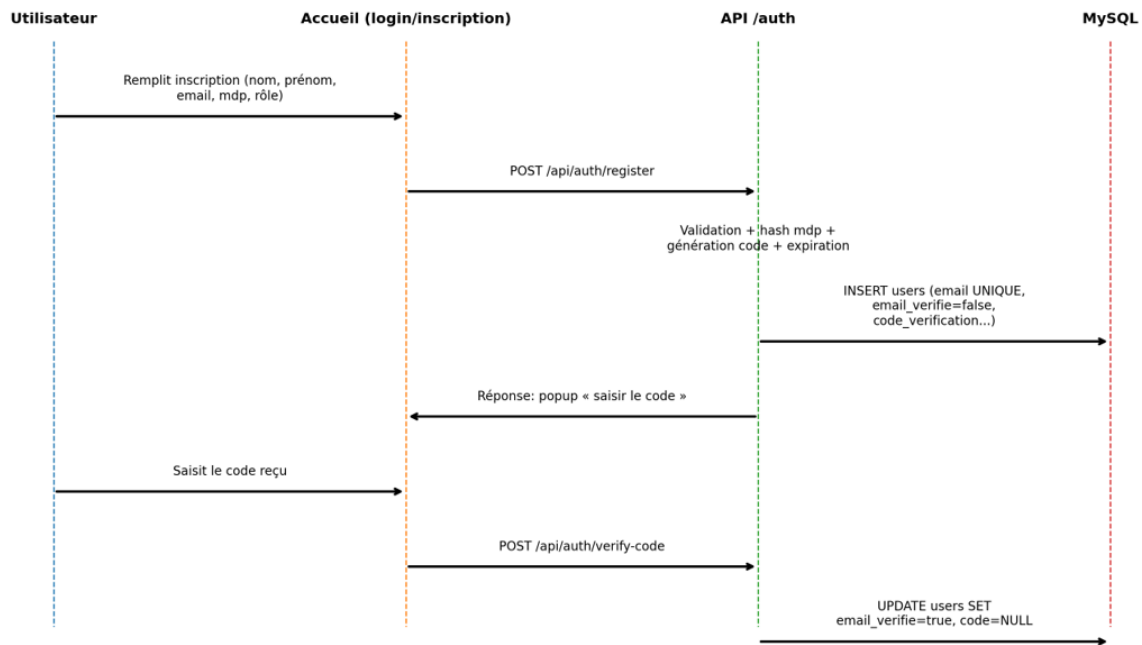


Figure 3: séquence inscriptions/vérification

Formulaire 3 : Vérification Email

Après l'inscription, un code à 6 chiffres est envoyé par email. L'utilisateur doit saisir ce code pour activer son compte.

Champ	Type HTML	Type	Requis	Placeholder	Validation
Code vérification	input	text	Oui	123456	6 chiffres exactement
Vérifier	button	submit	-	-	POST /api/auth/verify-code
Renvoyer code	button	secondary	-	-	POST /api/auth/send-code

Sécurité du code de vérification :

- Code génère cryptographiquement (module secrets)
- Expiration après 15 minutes (configurable DUREE_CODE_VÉRIFICATION)
- Possibilité de renvoyer un nouveau code (délai 1 minute)

Vérification Email

Un code à 6 chiffres a été envoyé à votre adresse email.

sofiaben28@gmail.com

Code de vérification

 RENVOYER  VALIDER

Bonjour,

Votre code de vérification est : 757520

Ce code est valable pendant 15 minutes.

Cordialement,
L'équipe IUT Lyon 1 GEII

Figure 4: Vérification de l'email

2.1.2 Dashboard Administrateur (dashboard_admin.html)

Rôle de la page : Interface complétée de gestion pour les administrateurs. Permet la gestion du matériel, des utilisateurs, des emprunts, des maintenances et l'accès aux statistiques et exports.

URL d'accès : <http://localhost:5000/dashboard-admin> (protégée par session)

Structure de la page :

La page est organisée avec une sidebar de navigation à gauche (260px) et une zone de contenu principale à droite. Le header contient le nom de l'utilisateur connecté et le toggle de thème clair/sombre.

Section	Description	Fonctionnalite principale
Dashboard	Vue d'ensemble avec KPI et graphiques statistiques	Première section affichée
Matériel	Liste, ajout, modification, suppression de matériel	CRUD complet avec photos
Emprunts	Gestion des emprunts en cours et historique	Validation retours
Maintenances	Suivi des maintenances planifiées et en cours	Assignment techniciens
Utilisateurs	Gestion des comptes utilisateurs	CRUD + activation/désactivation
Exports	Génération de rapports Excel et PDF	8 types d'exports

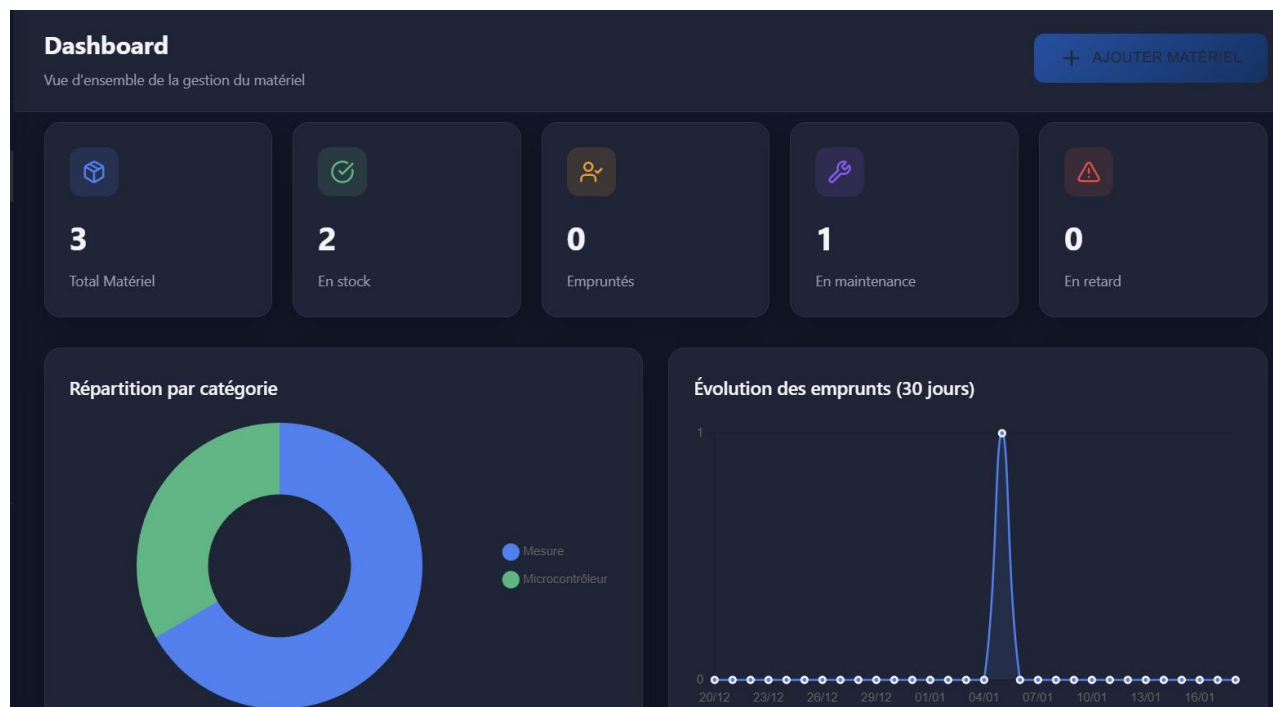


Figure 5: Dashboard admin

Section Dashboard : Cartes KPI

KPI	Description	Requête SQL
Total Matériel	Nombre total d'équipements dans la base	SELECT COUNT(*) FROM matériel
En Stock	Matériels disponibles pour emprunt	WHERE état = 'en_stock'
Emprunts Actifs	Emprunts en cours non-retournés	WHERE état = 'emprunte'
En Maintenance	Matériels chez les techniciens	WHERE état = 'en_maintenance'
Retards	Emprunts avec date retour dépassée	WHERE date_retour_prévue < CURDATE()

Section Dashboard : Graphiques Chart.js

Deux graphiques sont affichés pour visualiser les données statistiques :

1. Graphique Camembert (Doughnut) : Répartition du matériel par catégorie

- Données : SELECT catégories, COUNT(*) GROUP BY catégories
- Couleurs : Palette de 10 couleurs distinctes
- Interaction : Click sur segment pour filtrer la liste

2. Graphique Ligne (Line) : Evolution des emprunts sur 30 jours

- Données : COUNT par jour sur les 30 derniers jours
- Axe X : Dates (format DD/MM)
- Axe Y : Nombre d'emprunts
- Interaction : Tooltip au survol avec détails

Formulaire : Ajout de Matériel

Champ	Type	Format	Requis	Exemple/Note
Nom	input	text	Oui	Ex: Oscilloscope Rigol DS1054Z
Catégories	select	text	Non	Liste dynamique depuis BDD
Localisation	input	text	Non	Ex: Salle TP Électronique B-04
Description	textarea	text	Non	Reference, numéro série, notes
Photos	input	file	Non	Max 5 fichiers, 5Mo chacun

Informations Matériel

Nom :

Arduino Uno

Catégorie :

Microcontrôleur

Localisation :

B-04

Description :

ABC456844

Date retour prévue :

-

Dernier mouvement :

18/01/2026 19:46

Photos

Aucune photo

↑

AJOUTER UNE PHOTO

Figure 6: ajout de photos via information matériel

Formulaire : Modification d'État (Emprunt/Maintenance)

Champ	Type	Format	Requis	Options/Note
Nouvel État	select	enum	Oui	en_stock, emprunte, en_maintenance
Personne	select	int	Conditionnel	Emprunteur OU Technicien selon état
Date Retour	input	date	Conditionnel	Obligatoire si état=emprunte

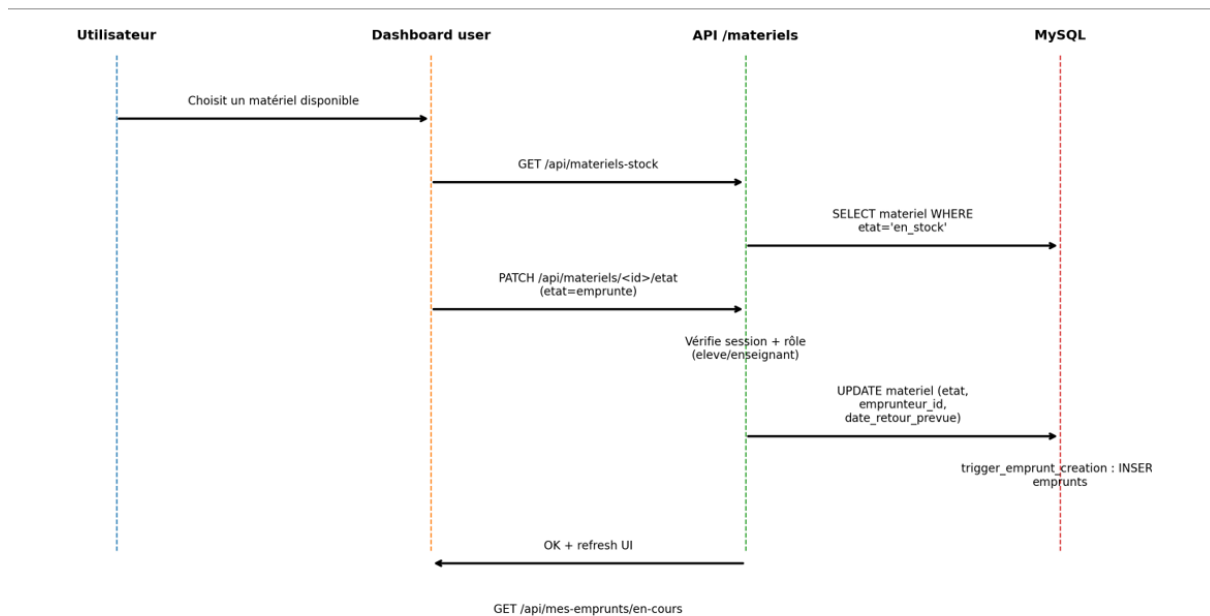


Figure 7: séquence emprunt

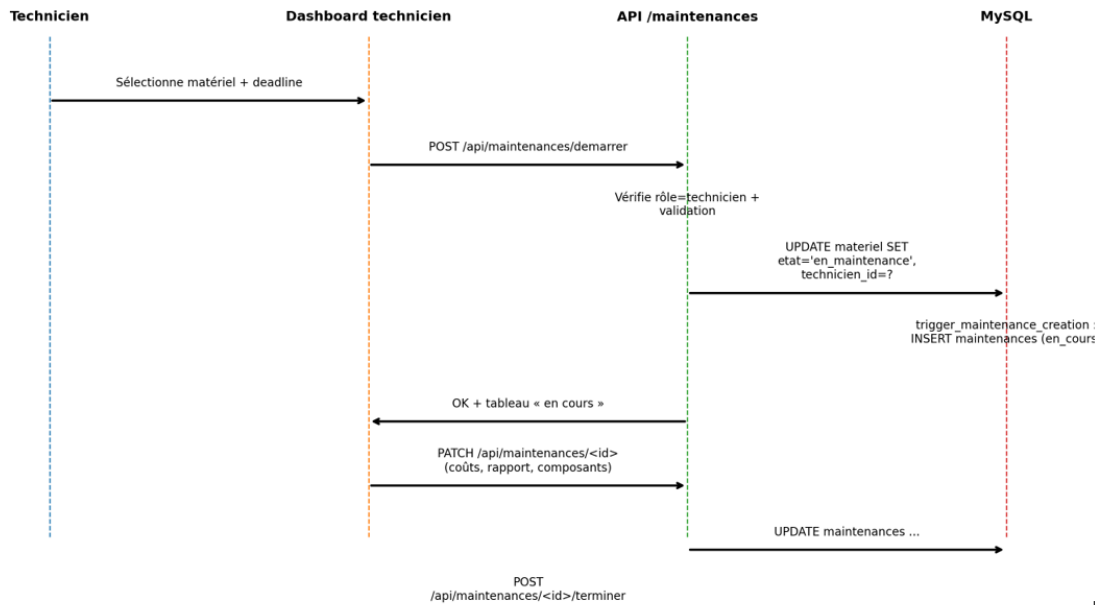


Figure 8: séquence maintenance

Logique métier du changement d'état :

- en_stock -> emprunte : Cree automatiquement un emprunt (trigger SQL)
- emprunte -> en_stock : Clôture l'emprunt avec date retour effective
- en_stock -> en_maintenance : Cree automatiquement une maintenance
- en_maintenance -> en_stock : Termine la maintenance

2.1.3 Dashboard Utilisateur (dashboard_user.html)

Rôle de la page : Interface simplifiée pour les élèves et enseignants. Permet de consulter le matériel disponible, visualiser ses emprunts en cours et son historique.

URL d'accès : http://localhost:5000/dashboard-user (protégée par session)

Rôles autorisés : élève, enseignant

Section	Description	Note
Matériel Disponible	Liste des matériels en stock disponibles à l'emprunt	Lecture seule
Mes Emprunts	Emprunts en cours de l'utilisateur connecte	Vue personnelle
Historique	Historique des emprunts passés (retournes)	50 derniers max
Retards	Liste des emprunts rendus en retard	Indicateur de fiabilité

Endpoints API utilisés :

- GET /api/matériels : Liste tous les matériels
- GET /api/mes-emprunts/en-cours : Emprunts actifs de l'utilisateur
- GET /api/mes-emprunts/historique : Emprunts terminés
- GET /api/mes-emprunts/retards : Emprunts rendus en retard

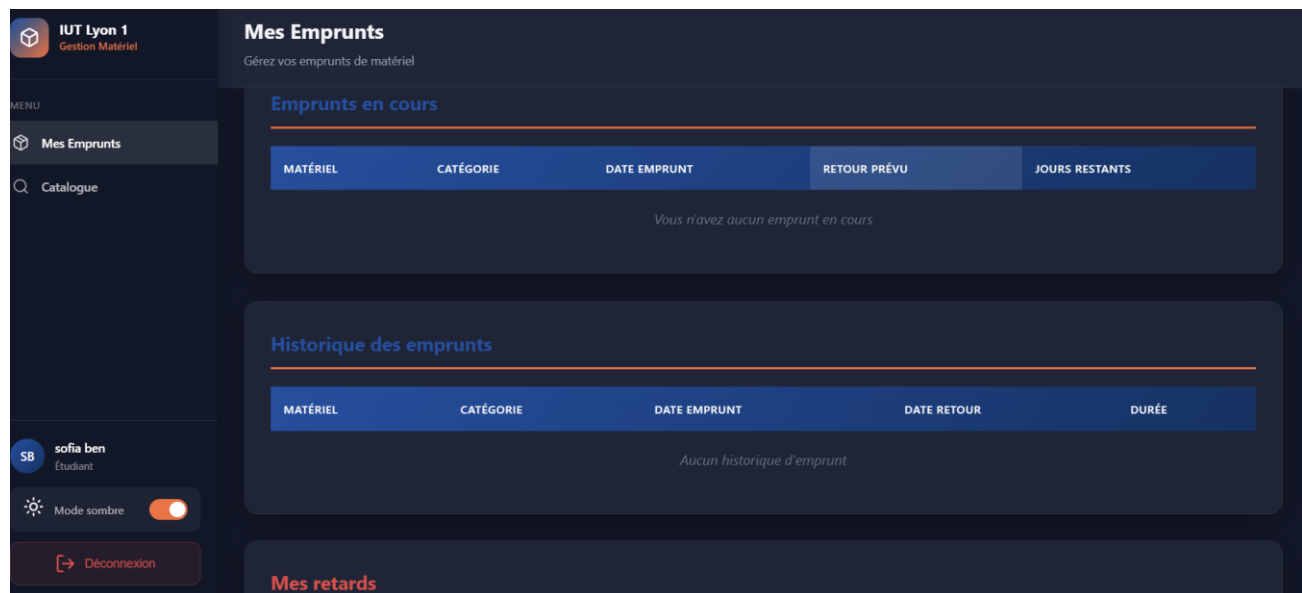


Figure 6: dashboard utilisateur (élève / enseignant)

2.1.4 Dashboard Technicien (dashboard_technicien.html)

Rôle de la page : Interface spécialisée pour les techniciens de maintenance. Permet de gérer les maintenances assignées, démarrer de nouvelles maintenances et suivre l'historique des interventions.

URL d'accès : http://localhost:5000/dashboard-technicien (protégée par session)

Section	Description	Note
Maintenances en Cours	Maintenances assignées au technicien connecte	Tri par deadline
Démarrer Maintenance	Formulaire pour prendre un matériel en maintenance	Depuis stock
Historique	Maintenances terminées par le technicien	Avec rapport et couts

Formulaire : Démarrer une Maintenance

Champ	Type	Format	Requis	Description
Matériel	select	int	Oui	Liste des matériels en_stock uniquement
Deadline	input	date	Oui	Date limite prévue pour terminer

Formulaire : Terminer une Maintenance

Champ	Type	Format	Requis	Description
Composants commandes	textarea	text	Non	Liste des pièces utilisées
Couts	input	number	Non	Montant en euros (ex: 45.50)
Rapport	textarea	text	Non	Description des travaux effectués

IUT Lyon 1
Gestion Matériel

Maintenances en cours
Gérez vos maintenances actives

Maintenances en cours

MATÉRIEL	CATÉGORIE	DATE DÉBUT	DEADLINE	JOURS RESTANTS
Keysight DSOX11002A	Mesure	17/01/2026 23:21	18/01/2026	0 JOURS

Ben Abdeslam Sofia
Technicien

Mode sombre

→ Déconnexion

Figure 7: Dashboard technicien de maintenance

2.2 Documentation de l'API REST

L'application expose une API REST (Representational State Transfer) pour la communication entre le frontend JavaScript et le backend Flask. Cette section documente tous les endpoints disponibles.

2.2.1 Vue d'Ensemble de l'API

Architecture de l'API :

- Base URL : `http://localhost:5000/api`
- Format : JSON (`application/json`)
- Authentification : Sessions Flask (cookie `HttpOnly`)
- Méthodes : GET, POST, PATCH, DELETE

Codes HTTP utilisés :

Code	Statut	Description
200	OK	Requête réussie
201	Created	Ressource créée avec succès
400	Bad Request	Données invalides ou manquantes
401	Unauthorized	Non authentifié
403	Forbidden	Accès refuse (droits insuffisants)
404	Not Found	Ressource introuvable
409	Conflict	Conflit (ex: email déjà utilise)
500	Internal Server Error	Erreur serveur

2.2.2 Endpoints Authentification (`/api/auth/*`)

POST `/api/auth/login`

Description : Connexion d'un utilisateur existant

Corps de la requête (JSON) :

```
{
  "email": "user@iut.fr",
  "password": "motdepasse"
}
```

Réponse succès (200) :

```
{
  "success": true,
  "rôle": "élève",
  "nom": "Jean Dupont"
}
```

Réponses erreur :

- 400 : Email et mot de passe requis
- 401 : Email ou mot de passe incorrect
- 403 : Email non vérifiée OU compte désactivé

POST /api/auth/register

Description : Inscription d'un nouvel utilisateur

Corps de la requête (JSON) :

```
{
  "nom": "Dupont",
  "prénom": "Jean",
  "email": "jean.dupont@iut.fr",
  "password": "motdepasse123",
  "rôle": "élève"
}
```

Réponse succès (201) :

```
{
  "success": true,
  "message": "Compte crée ! Vérifiez votre email.",
  "userId": 15,
  "email": "jean.dupont@iut.fr"
}
```

POST /api/auth/verify-code

Description : Vérification du code email

```
{
  "email": "jean.dupont@iut.fr",
  "code": "123456"
}
```

Autres endpoints authentication :

Méthode	Endpoint	Description	Body
POST	/api/auth/send-code	Renvoyer code vérification	{ email }
POST	/api/auth/logout	Déconnexion	-
GET	/api/auth/check-session	Vérifier session active	-

2.2.3 Endpoints Matériel (/api/matériels/*)

Méthode	Endpoint	Description	Body	Réponse
GET	/api/matériels	Liste tous les matériels	-	Array de matériels
POST	/api/matériels	Ajouter un matériel	{ nom, catégories, ... }	{ success, id }
PATCH	/api/matériels/:id	Modifier un matériel	{ nom, catégories, ... }	{ success }
DELETE	/api/matériels/:id	Supprimer un matériel	-	{ success }
PATCH	/api/matériels/:id/état	Changer l'état	{ état, personned, ... }	{ success }
GET	/api/matériels/:id/photos	Lister photos	-	Array de photos
POST	/api/matériels/:id/photos	Ajouter photo	FormData (photo)	{ success, chemin }
DELETE	/api/matériels/:id/photos/:photoid	Supprimer photo	-	{ success }

Détail : GET /api/matériels

Retourne la liste complétée des matériels avec informations jointes :

```
[
  {
    "id": 1,
    "nom": "Oscilloscope Rigol DS1054Z",
    "catégories": "Mesure",
    "état": "en_stock",
    "localisation": "Salle B-04",
    "description": "REF-2024-001",
    "date_ajout": "15/01/2026 10:30",
    "emprunteur_id": null,
    "emprunteur_nom_complet": null,
    "technicien_id": null,
    "jours_retard": 0,
    "nb_photos": 2
  },
  ...
]
```

2.2.4 Endpoints Utilisateurs (/api/users/*)

Méthode	Endpoint	Description	Body	Accès
GET	/api/users	Liste users actifs (pour selects)	-	admin,tech
GET	/api/users/all	Liste TOUS les users	-	admin
PATCH	/api/users/:id/toggle-actif	Activer/Désactiver	-	admin
PATCH	/api/users/:id/rôle	Modifier rôle	{ rôle }	admin
DELETE	/api/users/:id	Supprimer user	-	admin

2.2.5 Endpoints Emprunts (/api/mes-emprunts/*)

Méthode	Endpoint	Description	Accès requis
GET	/api/mes-emprunts/en-cours	Emprunts actifs du user connecte	Connecte
GET	/api/mes-emprunts/historique	Emprunts terminés (50 derniers)	Connecte
GET	/api/mes-emprunts/retards	Emprunts rendus en retard	Connecte

2.2.6 Endpoints Maintenances (/api/maintenances/*)

Méthode	Endpoint	Description	Accès
GET	/api/mes-maintenances/en-cours	Maintenances actives du technicien	technicien
GET	/api/mes-maintenances/historique	Maintenances terminées	technicien
POST	/api/maintenances/démarrer	Démarrer nouvelle maintenance	technicien
PATCH	/api/maintenances/:id	Modifier (rapport, couts)	technicien
POST	/api/maintenances/:id/terminer	Terminer et remettre en stock	technicien
GET	/api/matériels/:id/maintenances	Historique maintenances matériel	connecte

2.2.7 Endpoints Statistiques et Exports

Méthode	Endpoint	Description	Format sortie
GET	/api/stats/dashboard	KPI + données graphiques	JSON complet
GET	/api/export/inventaire	Export inventaire matériel	Excel (.xlsx)
GET	/api/export/élèves	Export liste élèves	Excel (.xlsx)
GET	/api/export/enseignants	Export liste enseignants	Excel (.xlsx)
GET	/api/export/techniciens	Export liste techniciens	Excel (.xlsx)
GET	/api/export/emprunts	Export historique emprunts	Excel (.xlsx)
GET	/api/export/maintenances	Export historique maintenances	Excel (.xlsx)
GET	/api/export/retards	Export retards en cours	Excel (.xlsx)
GET	/api/export/rapport-mensuel	Rapport mensuel complet	PDF (.pdf)

2.3 Description du Découpage en Dossiers/Fichiers

2.3.1 Structure Globale du Projet

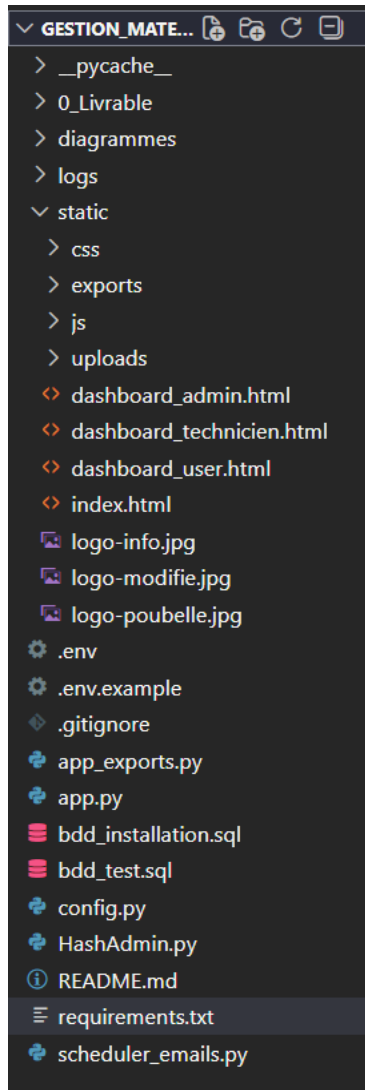
Le projet suit une architecture claire séparant le backend (Python/Flask), le frontend (HTML/CSS/JS) et les ressources statiques. Cette organisation facilite la maintenance et l'évolution du code.

Arborescence complétée :

```
Gestion_MatérielAG/
|
|-- app.py                # Application Flask principale (routes API)
|-- config.py             # Configuration (BDD, constantes, messages)
|-- app_exports.py        # Fonctions d'export Excel/PDF
|-- bdd.sql               # Schema SQL complet (tables, vues, triggers)
|-- requirements.txt       # Dépendances Python
|-- .env                  # Variables environnement (SECRETS - non versionné)
|-- .env.example          # Template des variables environnement
|
|-- static/               # Fichiers statiques (servis par Flask)
|   |-- index.html        # Page login/inscription
|   |-- dashboard_admin.html # Dashboard administrateur
|   |-- dashboard_user.html  # Dashboard élève/enseignant
|   |-- dashboard_technicien.html # Dashboard technicien
|   |
|   |-- css/
|   |   |-- style.css     # Styles CSS (thèmes, composants, responsive)
|   |
|   |-- js/
|   |   |-- auth.js       # Logique authentification (login, register)
|   |   |-- admin.js      # Logique dashboard admin
|   |   |-- user.js       # Logique dashboard user
|   |   |-- technicien.js  # Logique dashboard technicien
|   |   |-- thème.js      # Gestion thème clair/sombre
|   |   |-- toast.js      # Système notifications toast
|   |
|   |-- uploads/          # Photos matériels uploadées
|   |   |-- 1_20260115_photo.jpg
|   |   |-- ...
|   |
|-- logs/                 # Fichiers de logs
|   |-- app.log           # Logs application Flask
|
|-- diagrammes/           # Diagrammes UML PlantUML
|   |-- modèle_entité_association.puml
|   |-- schema_relationnel.puml
|   |-- architecture_application.puml
|
|-- exports/              # Fichiers exports générés (temporaire)
|   |-- inventaire_20260118.xlsx
|   |-- rapport_mensuel_202601.pdf
```

2.3.2 Description Détaillée des Fichiers Principaux

figure 8 : arborescence du projet



app.py - Application Flask Principale

Section	Contenu	Détails
Lignes 1-60	Imports et configuration	Flask, bcrypt, logging, dotenv
Lignes 61-103	Décorateur @handle_errors	Gestion centralisée des erreurs
Lignes 104-125	Configuration Flask	Secret key, SMTP, dossiers
Lignes 126-230	Fonctions utilitaires	Vérification fichier, envoi email, formatage dates
Lignes 231-270	Routes pages HTML	Redirection selon rôle
Lignes 271-585	API Authentification	/api/auth/* (login, register, verify)
Lignes 586-950	API Matériel	/api/matériels/* (CRUD, photos, états)
Lignes 951-1190	API Utilisateurs	/api/users/* (liste, toggle, rôle, delete)
Lignes 1191-1285	API Emprunts User	/api/mes-emprunts/* (en-cours, historique)

Lignes 1286-1520	API Maintenances	/api/maintenances/* (CRUD technicien)
Lignes 1521-1650	API Statistiques	/api/stats/dashboard (KPI, graphiques)
Lignes 1651-1705	Routes Exports	/api/export/* (délégation à app_exports.py)
Lignes 1706-1865	Système Emails	Rappels automatiques retards
Lignes 1866-1901	Lancement Serveur	Scheduler + Flask debug mode

config.py - Configuration Centralisée

Ce fichier centralise toutes les constantes et paramètres de l'application :

- Connexion BDD : get_db_connection() avec mysql-connector
- Constantes temporelles : DUREE_CODE_VÉRIFICATION, DUREE_EMPRUNT_DEFAULT
- Constantes pagination : ITEMS_PAR_PAGE
- Constantes fichiers : DOSSIER_UPLOAD, EXTENSIONS AUTORISEES, TAILLE_MAX
- Constantes emails : RAPPEL_AVANT_RETOUR, RAPPEL_RETARD_1/2/FACTURE
- Messages d'erreur : Dictionnaire ERREURS pour messages standardises

bdd.sql - Schema Base de Données

Section	Contenu	Détails
Lignes 1-15	Remise à zéro	DROP/CREATE DATABASE gestion_iut
Lignes 16-35	Table matériel	5 colonnes principales + FK
Lignes 36-55	Table users	11 colonnes + index
Lignes 56-75	Table emprunts	FK matériel + user
Lignes 76-96	Table maintenances	FK matériel + technicien
Lignes 97-110	Table photos_matériel	FK matériel + chemin
Lignes 111-140	Compte admin default	admin@iut.fr / M@teriel2025
Lignes 141-235	Vues SQL	3 vues (matériel, emprunts, maintenances)
Lignes 236-295	Triggers SQL	3 triggers (emprunt, retour, maintenance)

2.4 Modélisation de la Base de Données

2.4.1 Modèle Entité-Association (E/A)

Le modèle Entité-Association (E/A) est une méthode de conception qui permet de représenter les données et leurs relations avant l'implémentation physique. Ce modèle conceptuel est indépendant du SGBD utilise.

Légende du modèle E/A :

- Rectangle : Entité (table)
- Ovale : Attribut (colonne)
- Losange : Relation (association)
- Trait simple : Cardinalité 1
- Trait double : Cardinalité N (plusieurs)

Les 5 entités du modèle :

Entité	Description	Attributs principaux
USER	Utilisateurs du système	id, nom, prénom, email, password_hash, rôle, ...
MATÉRIEL	Équipements gérés	id, nom, catégories, état, localisation, ...
EMPRUNT	Historique emprunts	id, date_emprunt, date_retour_prévue, ...
MAINTENANCE	Interventions techniques	id, date_début, date_fin, couts, rapport, ...
PHOTO	Photos des matériels	id, chemin_photo, date_ajout

Les relations entre entités :

Entités liées	Relation	Cardinalité	Explication
USER - EMPRUNT	emprunte	1,N	1 user peut avoir N emprunts
MATÉRIEL - EMPRUNT	concerne	1,N	1 matériel peut être emprunte N fois
USER - MAINTENANCE	effectue	1,N	1 technicien effectue N maintenances
MATÉRIEL - MAINTENANCE	subit	1,N	1 matériel peut avoir N maintenances
MATÉRIEL - PHOTO	possède	1,N	1 matériel peut avoir N photos

DIAGRAMME UML 01

Le diagramme E/A est disponible dans le dépôt git

2.4.2 Schema Relationnel

Le schéma relationnel est la traduction du modèle E/A en tables SQL. Les entités deviennent des tables, les attributs deviennent des colonnes, et les relations sont implémentées via des clés étrangères.

Notation utilisée :

- # : Clé primaire (PRIMARY KEY)
- => : Clé étrangère (FOREIGN KEY)
- CK : Contrainte d'unicité (UNIQUE)

Schema relationnel complet :

```
users(#id, nom, prénom, email, password_hash, rôle, email_verifie,  
      code_vérification, date_code_expiration, date_création,  
      date_derniere_connexion, actif)  
CK: email UNIQUE  
  
matériel(#id, nom, catégories, état, localisation, description,  
         date_ajout, date_mouvement, date_retour_prévue,  
         =>emprunteur_id, =>technicien_id)  
FK: emprunteur_id REFERENCES users(id) ON DELETE SET NULL  
FK: technicien_id REFERENCES users(id) ON DELETE SET NULL  
  
emprunts(#id, =>matériel_id, =>emprunteur_id, date_emprunt,  
         date_retour_prévue, date_retour_effective, jours_retard, commentaire)  
FK: matériel_id REFERENCES matériel(id) ON DELETE CASCADE  
FK: emprunteur_id REFERENCES users(id) ON DELETE CASCADE  
  
maintenances(#id, =>matériel_id, =>technicien_id, date_début, date_fin,  
             deadline, composants_commandes, couts, rapport, statut)  
FK: matériel_id REFERENCES matériel(id) ON DELETE CASCADE  
FK: technicien_id REFERENCES users(id) ON DELETE CASCADE  
  
photos_matériel(#id, =>matériel_id, chemin_photo, date_ajout)  
FK: matériel_id REFERENCES matériel(id) ON DELETE CASCADE
```

DIAGRAMME UML 02

Le schéma relationnel est disponible dans le dépôt git

2.4.3 Description Détaillée des Tables SQL

Table : users

Rôle : Stocke les informations de tous les utilisateurs du système.

Colonne	Type	Contrainte	Description
id	INT	AUTO_INCREMENT PRIMARY KEY	Identifiant unique
nom	VARCHAR(100)	NOT NULL	Nom de famille
prénom	VARCHAR(100)	NOT NULL	Prénom
email	VARCHAR(255)	UNIQUE NOT NULL	Adresse email (login)
password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt du mot de passe
rôle	ENUM	NOT NULL	admin, élève, enseignant, technicien
email_verifie	BOOLEAN	DEFAULT 0	Email confirme ?
code_vérification	VARCHAR(6)	NULL	Code 6 chiffres (temporaire)
date_code_expiration	TIMESTAMP	NULL	Expiration du code
date_création	TIMESTAMP	DEFAULT NOW()	Date inscription
date_derniere_connexion	TIMESTAMP	NULL	Dernière connexion
actif	BOOLEAN	DEFAULT 1	Compte actif ?

Code SQL de création :

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nom VARCHAR(100) NOT NULL,  
  prénom VARCHAR(100) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  rôle ENUM('admin', 'élève', 'enseignant', 'technicien') NOT NULL,  
  email_verifie BOOLEAN DEFAULT 0,  
  code_vérification VARCHAR(6) NULL,  
  date_code_expiration TIMESTAMP NULL,  
  date_création TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  date_derniere_connexion TIMESTAMP NULL,  
  actif BOOLEAN DEFAULT 1,  
  INDEX idx_email (email),  
  INDEX idx_rôle (rôle)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Table : matériel

Rôle : Stocke les informations de tous les équipements gérés.

Colonne	Type	Contrainte	Description
id	INT	AUTO_INCREMENT PRIMARY KEY	Identifiant unique
nom	VARCHAR(255)	NOT NULL	Nom du matériel
catégories	VARCHAR(100)	NULL	Catégories (Mesure, Électronique, ...)
état	ENUM	DEFAULT 'en_stock'	en_stock, emprunte, en_maintenance
localisation	VARCHAR(100)	NULL	Emplacement physique
description	TEXT	NULL	Description, référence, notes
date_ajout	TIMESTAMP	DEFAULT NOW()	Date ajout au système
date_mouvement	TIMESTAMP	ON UPDATE NOW()	Dernier changement d'état
date_retour_prévue	DATE	NULL	Date retour si emprunte
emprunteur_id	INT	NULL FK users	ID emprunteur actuel
technicien_id	INT	NULL FK users	ID technicien si maintenance

Table : emprunts

Rôle : Historique complet de tous les emprunts passés et en cours.

Colonne	Type	Contrainte	Description
id	INT	AUTO_INCREMENT PRIMARY KEY	Identifiant unique
matériel_id	INT	NOT NULL FK matériel	Matériel emprunte
emprunteur_id	INT	NOT NULL FK users	Utilisateur emprunteur
date_emprunt	TIMESTAMP	DEFAULT NOW()	Date de début emprunt
date_retour_prévue	DATE	NOT NULL	Date retour prévue
date_retour_effective	TIMESTAMP	NULL	Date retour réel
jours_retard	INT	DEFAULT 0	Nombre jours de retard
commentaire	TEXT	NULL	Notes sur l'emprunt

Table : maintenances

Rôle : Suivi des interventions de maintenance sur le matériel.

Colonne	Type	Contrainte	Description
id	INT	AUTO_INCREMENT PRIMARY KEY	Identifiant unique
matériel_id	INT	NOT NULL FK matériel	Matériel en maintenance
technicien_id	INT	NOT NULL FK users	Technicien assigne
date_début	TIMESTAMP	DEFAULT NOW()	Date début intervention
date_fin	TIMESTAMP	NULL	Date fin (si terminée)
deadline	DATE	NOT NULL	Date limite prévue
composants_commandes	TEXT	NULL	Pieces commandées
couts	DECIMAL(10,2)	DEFAULT 0	Cout total en euros
rapport	TEXT	NULL	Rapport d'intervention
statut	ENUM	DEFAULT 'en_cours'	en_cours, terminée

2.4.4 Vues SQL

Les vues SQL sont des tables virtuelles définies par des requêtes SELECT. Elles simplifient les requêtes complexes en encapsulant les jointures.

Vue : vue_matériel_complet

Objectif : Retourner les matériels avec les noms des emprunteurs/techniciens.

```
CREATE OR REPLACE VIEW vue_matériel_complet AS
SELECT
    m.id, m.nom, m.catégories, m.état, m.localisation, m.description,
    m.date_ajout, m.date_mouvement, m.date_retour_prévue,
    m.emprunteur_id,
    CONCAT(e.prénom, ' ', e.nom) AS emprunteur_nom_complet,
    e.email AS emprunteur_email,
    m.technicien_id,
    CONCAT(t.prénom, ' ', t.nom) AS technicien_nom_complet,
    CASE
        WHEN m.date_retour_prévue < CURDATE() THEN DATEDIFF(CURDATE(), m.date_retour_prévue)
        ELSE 0
    END AS jours_retard,
    (SELECT COUNT(*) FROM photos_matériel p WHERE p.matériel_id = m.id) AS nb_photos
FROM matériel m
LEFT JOIN users e ON m.emprunteur_id = e.id
LEFT JOIN users t ON m.technicien_id = t.id;
```

Vue : vue_emprunts_utilisateur

Objectif : Afficher l'historique des emprunts avec statut calcule automatiquement.

```
CREATE OR REPLACE VIEW vue_emprunts_utilisateur AS
SELECT
    emp.id, emp.emprunteur_id,
    CONCAT(u.prénom, ' ', u.nom) AS emprunteur_nom,
    emp.matériel_id, m.nom AS matériel_nom,
    emp.date_emprunt, emp.date_retour_prévue, emp.date_retour_effective,
    emp.jours_retard,
    CASE
        WHEN emp.date_retour_effective IS NULL THEN 'en_cours'
        WHEN emp.jours_retard > 0 THEN 'retourne_en_retard'
        ELSE 'retourne_a_temps'
    END AS statut_emprunt,
    DATEDIFF(COALESCE(emp.date_retour_effective, CURDATE()), emp.date_emprunt) AS duree_jours
FROM emprunts emp
INNER JOIN users u ON emp.emprunteur_id = u.id
INNER JOIN matériel m ON emp.matériel_id = m.id;
```

Vue : vue_maintenances_complet

Objectif : Afficher les maintenances avec noms et calculs de durée.

2.4.5 Triggers SQL

Les triggers (déclencheurs) sont des blocs de code SQL qui s'exécutent automatiquement lors d'évènements sur les tables (INSERT, UPDATE, DELETE). Ils garantissent la cohérence des données.

Trigger : trigger_emprunt_création

Evènement : AFTER UPDATE sur matériel

Condition : Passage de tout état vers 'emprunte' avec emprunteur_id non null

Action : Cree automatiquement un enregistrement dans la table emprunts

```
DELIMITER //
CREATE TRIGGER trigger_emprunt_création
AFTER UPDATE ON matériel
FOR EACH ROW
BEGIN
    -- Si le matériel passe à l'état "emprunte" avec un emprunteur
    IF OLD.état != 'emprunte' AND NEW.état = 'emprunte'
    AND NEW.emprunteur_id IS NOT NULL THEN
        -- Créer l'enregistrement d'emprunt
        INSERT INTO emprunts (matériel_id, emprunteur_id, date_emprunt,
                               date_retour_prévue, jours_retard)
        VALUES (
            NEW.id,                -- ID du matériel
            NEW.emprunteur_id,     -- ID de l'emprunteur
            NOW(),                 -- Date actuelle
            COALESCE(NEW.date_retour_prévue, DATE_ADD(CURDATE(), INTERVAL 7 DAY)),
            0                      -- Pas de retard au départ
        );
    END IF;
END //
```

Avantage : L'admin n'a qu'à changer l'état du matériel, l'emprunt est créé automatiquement.

Trigger : trigger_emprunt_retour

Evènement : AFTER UPDATE sur matériel

Condition : Passage de 'emprunte' vers 'en_stock'

Action : Met à jour l'emprunt avec date_retour_effective et calcule le retard

```
DELIMITER //
CREATE TRIGGER trigger_emprunt_retour
AFTER UPDATE ON matériel
FOR EACH ROW
BEGIN
    -- Si le matériel passe de "emprunte" a "en_stock"
    IF OLD.état = 'emprunte' AND NEW.état = 'en_stock' THEN
        -- Mettre à jour le dernier emprunt (celui sans date de retour)
        UPDATE emprunts
        SET date_retour_effective = NOW(),
            jours_retard = CASE
                WHEN date_retour_prévue < CURDATE()
                THEN DATEDIFF(CURDATE(), date_retour_prévue)
                ELSE 0
            END
        WHERE matériel_id = NEW.id
        AND date_retour_effective IS NULL
        ORDER BY date_emprunt DESC
        LIMIT 1;
    END IF;
END //
DELIMITER ;
```

Trigger : trigger_maintenance_création

Evènement : AFTER UPDATE sur matériel

Condition : Passage vers 'en_maintenance' avec technicien_id non null

Action : Cree automatiquement un enregistrement dans la table maintenances

PARTIE 3 : MAINTENANCE ET EVOLUTIVITE

3.1 Procédures de Sauvegarde

La sauvegarde régulière des données est essentielle pour garantir la pérennité de l'application et la protection contre les pertes de données. Cette section détaille les procédures de sauvegarde et de restauration mises en place.

3.1.1 Importance des Sauvegardes

Raisons de sauvegarder régulièrement :

- Protection contre les pannes matérielles (disque dur défaillant)
- Protection contre les erreurs humaines (suppression accidentelle)
- Protection contre les attaques (ransomware, corruption de données)
- Conformité règlementaire (RGPD impose la protection des données)
- Continuité d'activité (reprise rapide après incident)

Risques en cas d'absence de sauvegarde :

Type incident	Description	Gravite	Temps reprise sans backup
Perte totale	Données non récupérables	Critique	Plusieurs semaines
Corruption BDD	Tables endommagées	Élève	Quelques jours
Erreur humaine	DELETE sans WHERE	Moyen	Quelques heures
Panne serveur	Disque HS	Élève	1-3 jours

3.1.2 Stratégie de Sauvegarde (3-2-1)

Nous appliquons la règle 3-2-1, reconnue comme la meilleure pratique en matière de sauvegarde :

3 copies des données (1 production + 2 sauvegardes)

2 types de supports différents (serveur local + cloud)

1 copie hors site (stockage externe/cloud)

Fréquence des sauvegardes :

Type	Fréquence	Heure	Contenu	Taille estimée
Complété	Quotidienne	02:00	Toute la base	~50 Mo
Incrémentale	Toutes les 4h	06:00, 10:00, 14:00, 18:00, 22:00	Modifications depuis dernière	~5 Mo
Avant mise à jour	Manuelle	Avant déploiement	Toute la base + code	~100 Mo

3.1.3 Commandes de Sauvegarde MySQL

Sauvegarde manuelle avec mysqldump :

```
# Sauvegarde complétée de la base gestion_iut
mysqldump -u root -p gestion_iut > backup_$(date +%Y%m%d_%H%M%S).sql

# Exemple de résultat : backup_20260118_143522.sql

# Options recommandées pour production :
mysqldump -u root -p \
  --single-transaction \
  --routines \
  --triggers \
  --add-drop-table \
  gestion_iut > backup_complet_$(date +%Y%m%d).sql
```

Explication des options :

--single-transaction : Garantit cohérence sans verrouiller les tables (InnoDB)

--routines : Inclut les procédures stockées et fonctions

--triggers : Inclut les triggers (déclencheurs)

--add-drop-table : Ajoute DROP TABLE avant CREATE (facilite restauration)

Script de sauvegarde automatisée (Linux) :

```
#!/bin/bash
# Fichier : /opt/scripts/backup_gestion_iut.sh

# Configuration
DB_USER="root"
DB_PASS="Gestion_IUT691"
DB_NAME="gestion_iut"
BACKUP_DIR="/var/backups/mysql"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="$BACKUP_DIR/${DB_NAME}_${DATE}.sql"
RÉTENTION_DAYS=30

# Créer le dossier si inexistant
mkdir -p $BACKUP_DIR

# Sauvegarde
echo "[$(date)] Début sauvegarde $DB_NAME"
mysqldump -u $DB_USER -p$DB_PASS \
  --single-transaction --routines --triggers \
  $DB_NAME > $BACKUP_FILE

# Compression (reduit taille de 80%)
gzip $BACKUP_FILE
echo "[$(date)] Sauvegarde compressée : ${BACKUP_FILE}.gz"

# Suppression des anciennes sauvegardes (> 30 jours)
find $BACKUP_DIR -name "*.sql.gz" -mtime +$RÉTENTION_DAYS -delete
```



```
echo "[$(date)] Nettoyage des sauvegardes > $RÉTENTION_DAYS jours"
```

```
# Vérification taille
```

```
du -sh $BACKUP_DIR
```

```
echo "[$(date)] Sauvegarde terminée avec succès"
```

Planification avec cron (Linux) :

```
# Editer le crontab
```

```
crontab -e
```

```
# Ajouter les lignes suivantes :
```

```
# Sauvegarde complétée tous les jours à 2h du matin
```

```
0 2 * * * /opt/scripts/backup_gestion_iut.sh >> /var/log/backup.log 2>&1
```

```
# Sauvegarde incrémentale toutes les 4 heures
```

```
0 6,10,14,18,22 * * * /opt/scripts/backup_incremental.sh >> /var/log/backup.log 2>&1
```

Format cron : minute heure jour_mois mois jour_semaine commande

3.1.4 Procédures de Restauration

La restauration permet de récupérer les données à partir d'une sauvegarde. Il est essentiel de tester régulièrement les restaurations pour s'assurer que les sauvegardes sont fonctionnelles.

Commandes de restauration :

```
# Décompresser la sauvegarde si nécessaire
gunzip backup_gestion_iut_20260118.sql.gz

# Restauration complétée (ATTENTION : écrase les données existantes)
mysql -u root -p gestion_iut < backup_gestion_iut_20260118.sql

# Restauration vers une nouvelle base (test avant production)
mysql -u root -p -e "CREATE DATABASE gestion_iut_restore;"
mysql -u root -p gestion_iut_restore < backup_gestion_iut_20260118.sql

# Vérification après restauration
mysql -u root -p gestion_iut -e "SELECT COUNT(*) FROM users;"
mysql -u root -p gestion_iut -e "SELECT COUNT(*) FROM matériel;"
mysql -u root -p gestion_iut -e "SHOW TABLES;"
```

IMPORTANT - SECURITE

Toujours tester la restauration sur une base de test avant de restaurer en production. Une mauvaise manipulation peut entraîner une perte de données.

Procédure de restauration complétée (étape par étape) :

Etape	Action	Commande	Remarque
1	Arrêter l'application	systemctl stop gestion-matériel	Evite conflits
2	Sauvegarder l'état actuel	mysqldump > backup_avant_restore.sql	Sécurité
3	Identifier la sauvegarde	ls -la /var/backups/mysql/	Choisir la bonne date
4	Décompresser	gunzip backup_YYYYMMDD.sql.gz	Si compressé
5	Restaurer	mysql < backup_YYYYMMDD.sql	Importer les données
6	Vérifier	mysql -e 'SELECT COUNT(*)...'	Contrôler intégrité
7	Redémarrer	systemctl start gestion-matériel	Relancer l'application
8	Tester	Navigateur http://localhost:5000	Vérification fonctionnelle

3.2 Maintenance Applicative

La maintenance applicative comprend la gestion des logs, les mises à jour du code et des dépendances, ainsi que le monitoring de la sante du système. Ces opérations garantissent la stabilité et la sécurité de l'application.

3.2.1 Gestion des Logs

Types de logs générés par l'application :

Fichier	Emplacement	Source	Contenu
app.log	logs/	Application Flask	Requêtes, erreurs, actions utilisateurs

Configuration du logging dans app.py :

```
import logging
from logging.handlers import RotatingFileHandler
import os

# Créer le dossier logs si inexistant
if not os.path.exists('logs'):
    os.makedirs('logs')

# Configuration du logger
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)

# Handler pour fichier avec rotation
file_handler = RotatingFileHandler(
    'logs/app.log',
    maxBytes=10*1024*1024, # 10 Mo par fichier
    backupCount=5          # Garde les 5 derniers fichiers
)
file_handler.setLevel(logging.INFO)
file_handler.setFormatter(logging.Formatter(
    '%(asctime)s - %(levelname)s - %(message)s'
))

app.logger.addHandler(file_handler)
```

Rotation des logs :

- Taille max par fichier : 10 Mo
- Nombre de fichiers conservés : 5 (app.log, app.log.1, ..., app.log.5)
- Rotation automatique quand taille atteinte
- Ancien fichier renommé avec suffixe numérique

Commandes utiles pour analyser les logs :

```
# Afficher les 50 dernières lignes
tail -n 50 logs/app.log

# Suivre en temps réel (live)
tail -f logs/app.log

# Rechercher les erreurs
grep -i "error" logs/app.log

# Compter les erreurs par type
grep -i "error" logs/app.log | cut -d'-' -f4 | sort | uniq -c | sort -rn

# Filtrer par date (ex: 18 janvier 2026)
grep "2026-01-18" logs/app.log

# Trouver les requêtes les plus lentes (> 1 seconde)
grep "took [1-9][0-9]*\\.\\.s" logs/app.log
```

3.2.2 Mises à jour de l'Application

Types de mises à jour :

Composant	Outil	Fréquence	Raison	Commande
Dépendances Python	pip	Mensuelle	Sécurité, bugs	pip install --upgrade -r requirements.txt
Code applicatif	Git	Continue	Fonctionnalités	git pull origin main
Base de données	SQL	Au besoin	Schema, migrations	mysql < migration_vX.sql
Système d'exploitation	apt/yum	Mensuelle	Sécurité	apt update && apt upgrade

Procédure de mise à jour du code :

```
# 1. Sauvegarder avant mise à jour
mysqldump -u root -p gestion_iut > backup_avant_update_$(date +%Y%m%d).sql

# 2. Arrêter l'application
systemctl stop gestion-matériel # ou Ctrl+C si en développement

# 3. Récupérer les dernières modifications
cd /chemin/vers/Gestion_MatérielAG
git fetch origin
```

```

git status # Vérifier l'état actuel
git pull origin main

# 4. Mettre à jour les dépendances Python
pip install --upgrade -r requirements.txt

# 5. Appliquer les migrations SQL si nécessaire
mysql -u root -p gestion_iut < migrations/migration_v1.2.sql

# 6. Redémarrer l'application
systemctl start gestion-matériel # ou python app.py

# 7. Vérifier le bon fonctionnement
curl http://localhost:5000/api/auth/check-session

```

IMPORTANT - AVANT TOUTE MISE A JOUR

Toujours effectuer une sauvegarde complétée AVANT de mettre à jour. En cas de problème, vous pourrez restaurer l'état précédent.

3.2.3 Monitoring et Surveillance

Le monitoring permet de surveiller la sante de l'application en temps réel et d'être alerte en cas de problème (serveur inaccessible, erreurs fréquentes, espace disque insuffisant, etc.).

Indicateurs à surveiller (KPI techniques) :

Indicateur	Description	Seuil acceptable	Outil
Disponibilité	Uptime du serveur	> 99.5%	ping, curl
Temps de réponse	Latence des requêtes API	< 500ms	Logs, APM
Taux d'erreur	% requêtes en erreur (5xx)	< 1%	Logs, alertes
Espace disque	Utilisation du disque	< 80%	df -h
Mémoire	RAM utilisée	< 80%	free -m
CPU	Charge processeur	< 70%	top, htop
Connexions BDD	Connexions MySQL actives	< 100	SHOW PROCESSLIST

Script de monitoring basique :

```

#!/bin/bash
# Fichier : /opt/scripts/check_health.sh

# Couleurs pour l'affichage
RED='\033[0;31m'
GREEN='\033[0;32m'
NC='\033[0m'

echo "=== HEALTH CHECK - $(date) ==="

# 1. Vérifier que l'application répond
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:5000/)

```

```

if [ "$HTTP_CODE" == "200" ]; then
    echo -e "${GREEN}[OK]${NC} Application accessible (HTTP $HTTP_CODE)"
else
    echo -e "${RED}[ERREUR]${NC} Application inaccessible (HTTP $HTTP_CODE)"
fi

# 2. Vérifier la base de données
mysql -u root -pGestion_IUT691 -e "SELECT 1" gestion_iut > /dev/null 2>&1
if [ $? -eq 0 ]; then
    echo -e "${GREEN}[OK]${NC} Base de données accessible"
else
    echo -e "${RED}[ERREUR]${NC} Base de données inaccessible"
fi

# 3. Vérifier l'espace disque
DISK_USAGE=$(df -h / | awk 'NR==2 {print $5}' | sed 's/%//')
if [ "$DISK_USAGE" -lt 80 ]; then
    echo -e "${GREEN}[OK]${NC} Espace disque : ${DISK_USAGE}%"
else
    echo -e "${RED}[ALERTE]${NC} Espace disque critique : ${DISK_USAGE}%"
fi

# 4. Vérifier la mémoire
MEM_USAGE=$(free | awk 'NR==2 {printf "%.0f", $3/$2*100}')
if [ "$MEM_USAGE" -lt 80 ]; then
    echo -e "${GREEN}[OK]${NC} Mémoire : ${MEM_USAGE}%"
else
    echo -e "${RED}[ALERTE]${NC} Mémoire critique : ${MEM_USAGE}%"
fi

echo "======"

```

Configuration des alertes email :

En cas de problème détecté, le script peut envoyer une alerte :

- Application inaccessible -> Email admin + SMS
- Espace disque > 80% -> Email admin
- Erreurs fréquentes (> 10/heure) -> Email équipe technique

3.3 Evolutions Futures et Roadmap

Cette section présente les évolutions prévues pour les prochaines versions de l'application. Ces améliorations sont priorisées en fonction des besoins des utilisateurs et de la complexité technique.

3.3.1 Evolutions Court Terme (v1.1 - T1 2026)

Priorité haute - A livrer dans les 3 prochains mois :

ID	Fonctionnalite	Description	Complexité	Estimation
E01	QR Codes Matériel	Génération QR code unique par matériel. Scan avec smartphone pour afficher la fiche.	Moyenne	1-2 semaines
E02	Page Reset Password	Interface pour réinitialiser le mot de passe oublié (backend déjà fait)	Basse	2-3 jours
E03	Filtres avances	Ajout de filtres par date, par localisation dans la liste matériel	Basse	1 semaine
E04	Notifications in-app	Badge avec nombre de notifications non lues dans le header	Moyenne	1 semaine

3.3.2 Evolutions Moyen Terme (v2.0 - T2-T3 2026)

Priorité moyenne - A livrer dans les 6-9 mois :

ID	Fonctionnalite	Description	Complexité	Estimation
E05	Système de Réservations	Permettre de réserver un matériel à l'avance pour une date donnée	Elevée	3-4 semaines
E06	Notifications Push	Alertes temps réel via WebSocket (nouvel emprunt, retard, etc.)	Elevée	3 semaines
E07	Import CSV/Excel	Importer des matériels en masse depuis un fichier Excel	Moyenne	1-2 semaines
E08	API REST avec JWT	Authentification par token pour clients externes (apps mobiles)	Elevée	2-3 semaines
E09	Historique Audit	Log de toutes les actions utilisateurs (qui a fait quoi quand)	Moyenne	1 semaine

3.3.3 Evolutions Long Terme (v3.0 - 2027)

Vision à long terme - Fonctionnalités avancées :

ID	Fonctionnalite	Description	Complexité	Estimation
E10	Application Mobile	App iOS/Android native (React Native ou Flutter)	Très élevée	2-3 mois
E11	Dashboard BI	Intégration Power BI pour analyses avancées et prévisions	Très élevée	1-2 mois
E12	Multisites	Gestion de plusieurs départements/sites IUT	Elevée	1 mois
E13	Intelligence Artificielle	Prévision des pannes, suggestions d'achat basées sur usage	Très élevée	2-3 mois

3.3.4 Roadmap du Projet

Calendrier prévisionnel des releases :

Version	Date prévue	Contenu principal	Statut
v1.0	Janvier 2026	Version initiale complétée	Terminée
v1.1	Mars 2026	QR Codes + Reset Password + Filtres	Planifiée
v1.2	Mai 2026	Notifications in-app + Améliorations UX	Planifiée
v2.0	Septembre 2026	Réservations + Import CSV + API JWT	Planifiée
v2.5	Décembre 2026	Notifications Push + Audit Trail	Planifiée
v3.0	Juin 2027	App Mobile + Dashboard BI	Vision

Principes guident les évolutions :

1. Ecoute des utilisateurs : Priorité aux fonctionnalités les plus demandées
2. Stabilité avant nouveautés : Ne pas introduire de régressions
3. Tests avant déploiement : Chaque feature est testée en préproduction
4. Documentation à jour : Chaque évolution est documentée
5. Rétrocompatibilité : Les anciennes fonctionnalités restent opérationnelles

Autoévaluation

Note proposée : 17.5/ 20

Critère	Note/4	Commentaire
Conception BDD	4	Modélisation complète avec vues et triggers
Qualité du code	3.5	Architecture modulaire et commentaires
Interface utilisateur	4	Design responsive et intuitive
Documentation	3	Rapport complet et procédures détaillées
Tests et validation	3	Plan de tests complet avec preuves

Répartition des contributions

- **Chaalane Badice** : 30%
- **Ben abdeslam Sofia** : 30%
- **Hemici Séfana** : 15%
- **Samadi Chahd**: 15%

ANNEXES

Annexe A : Glossaire Technique

Ce glossaire définit les termes techniques utilisés dans ce document et dans l'application. Il est destiné aux utilisateurs et développeurs souhaitant comprendre le vocabulaire employé.

A.1 Termes Généraux

Terme	Signification	Définition
API	Application Programming Interface	Interface permettant à deux applications de communiquer entre elles
Backend	Partie serveur	Code exécute côté serveur (Python/Flask), gère la logique métier
Frontend	Partie client	Code exécute dans le navigateur (HTML/CSS/JS), gère l'interface
CRUD	Create Read Update Delete	Les 4 opérations de base sur les données
REST	Representational State Transfer	Architecture standard pour les APIs web
JSON	JavaScript Object Notation	Format de données léger utilisé pour les échanges API
SGBD	Système de Gestion de Base de Données	Logiciel gérant les bases (MySQL, PostgreSQL, etc.)

A.2 Termes Techniques

Terme	Signification	Définition
Bcrypt	Algorithme de hachage	Méthode sécurisée pour stocker les mots de passe (irréversible)
Cookie	Fichier stocké navigateur	Petit fichier permettant de maintenir la session utilisateur
Session	État utilisateur	Mécanisme pour garder un utilisateur connecté entre les pages
Token	Jeton d'authentification	Chaîne unique permettant d'identifier un utilisateur (JWT)
Hash	Empreinte numérique	Résultat d'une fonction de hachage, toujours même taille
Trigger	Déclencheur SQL	Code automatiquement exécuté lors d'un événement en base
Vue SQL	Table virtuelle	Requête SELECT enregistrée comme une table (pas de stockage)
Index	Index de base de données	Structure accélérant les recherches sur une colonne
FK	Foreign Key	Clé étrangère, référence vers une autre table
PK	Primary Key	Clé primaire, identifiant unique d'un enregistrement

A.3 Termes Metier (Application)

Terme	Signification	Définition
Emprunt	Pret de matériel	Action de confier un matériel a un utilisateur pour une durée
Retard	Dépassement délai	Emprunt non retourne après la date prévue
Maintenance	Intervention technique	Réparation ou vérification d'un matériel par un technicien
Stock	Matériel disponible	État d'un matériel prêt à être emprunte
KPI	Key Performance Indicator	Indicateur clé de performance (ex: taux de retard)
Dashboard	Tableau de bord	Interface affichant les métriques et actions principales
Export	Extraction données	Génération d'un fichier (Excel, PDF) à partir des données

Annexe B : Comptes de Test

Cette annexe liste les comptes utilisateurs créés pour les tests de l'application. Ces comptes permettent de vérifier le bon fonctionnement des différents rôles et permissions.

IMPORTANT - SECURITE

Ces comptes sont destinés aux TESTS uniquement. En production, il est impératif de changer tous les mots de passe par des mots de passe forts et uniques. Ne jamais utiliser ces identifiants en environnement de production.

B.1 Liste des Comptes de Test

Email	Mot de passe	Rôle	Statut	Utilisation
admin@iut.fr	M@teriel2025	admin	Actif	Compte administrateur principal avec tous les droits
technicien@iut.fr	M@teriel2025	technicien	Actif	Compte technicien pour tester les maintenances
enseignant@iut.fr	M@teriel2025	enseignant	Actif	Compte enseignant pour tester les emprunts
élève@iut.fr	M@teriel2025	élève	Actif	Compte élève pour tester la consultation

B.2 Permissions par Rôle

Matrice des permissions selon le rôle :

Action	Admin	Technicien	Enseignant	Élève
Consulter matériel	Oui	Oui	Oui	Oui
Voir ses emprunts	Oui	Oui	Oui	Oui
Créer emprunt	Oui (pour tous)	Non	Non	Non
Gérer matériel (CRUD)	Oui	Non	Non	Non
Gérer utilisateurs	Oui	Non	Non	Non
Démarrer maintenance	Oui	Oui	Non	Non
Terminer maintenance	Oui	Oui	Non	Non
Voir statistiques	Oui	Non	Non	Non
Exporter données	Oui	Non	Non	Non
Changer thème	Oui	Oui	Oui	Oui

B.3 Scenarios de Test Recommandes

Scenarios a exécuter pour valider le bon fonctionnement :

1. Test connexion Admin :

- Se connecter avec admin@iut.fr / M@teriel2025

- Vérifier redirection vers dashboard_admin.html
- Vérifier accès à toutes les sections (Matériel, Users, Stats, Exports)

2. Test connexion Technicien :

- Se connecter avec technicien@iut.fr / M@teriel2025
- Vérifier redirection vers dashboard_technicien.html
- Vérifier possibilité de démarrer et terminer une maintenance

3. Test connexion Élève :

- Se connecter avec élève@iut.fr / M@teriel2025
- Vérifier redirection vers dashboard_user.html
- Vérifier consultation du matériel (lecture seule)
- Vérifier impossibilité d'accéder au dashboard admin (403)

4. Test compte inactif :

- Tenter de se connecter avec test.inactif@iut.fr / M@teriel2025
- Vérifier message d'erreur 'Compte désactivé'

Annexe C : Contacts et Support

Cette annexe fournit les coordonnées des personnes à contacter en cas de problème technique, de question sur l'utilisation de l'application, ou pour toute demande d'évolution.

C.1 Equipe Projet

Rôle	Nom	Domaine
Développeurs principaux	Chaalane / ben abdeslam/hemici/samadi	Questions techniques, bugs
Encadrant SAE	P.Bouron	Questions pédagogiques
Responsable département	C.Lacharmoise	Décisions stratégiques

C.2 Support Technique

En cas de problème technique, suivre cette procédure :

1. Consulter la documentation :

- Ce document (Document de Conception et Maintenance)
- Le fichier README.md a la racine du projet
- La documentation d'installation (Document_Installation.docx)

2. Vérifier les logs :

- Consulter logs/app.log pour les erreurs récentes
- Identifier le message d'erreur exact

3. Signaler le problème :

- Ouvrir une issue sur le GitLab IUT (si disponible)
- OU envoyer un email au développeur avec :
 - * Description du problème
 - * Etapes pour reproduire
 - * Capture d'écran si possible
 - * Extrait du fichier log

C.3 Ressources Utiles

Liens vers la documentation externe :

Technologie	URL	Description
Flask (Python)	https://flask.palletsprojects.com/	Framework web backend
MySQL	https://dev.mysql.com/doc/	Documentation base de données
Chart.js	https://www.chartjs.org/docs/	Librairie graphiques JavaScript
Lucide Icons	https://lucide.dev/icons/	Icones utilisées dans l'interface
Bootstrap (inspiration)	https://getbootstrap.com/docs/	Framework CSS référence

C.4 Demandes d'Evolution

Pour proposer une nouvelle fonctionnalité ou une amélioration, contactez le développeur ou l'encadrant avec les informations suivantes :

- Description de la fonctionnalité souhaitée
- Cas d'usage concret (pourquoi cette fonctionnalité est utile)
- Utilisateurs concernés (admin, technicien, élève, tous)
- Priorité souhaitée (critique, haute, moyenne, basse)
- Maquette ou schéma si possible

Les demandes seront évaluées et intégrées à la roadmap selon leur priorité et la charge de travail disponible.